



Monorepos

The Benefits, Challenges & Importance of Tooling Support

Juri Strumpflohner · @juristr · Tech Talks South Tyrol · Feb 2024



Juri Strumpflohner

Sr. Director of Developer
Experience 

@juristr



Google Developers Expert

Web Technologies



Instructor

egghead.io





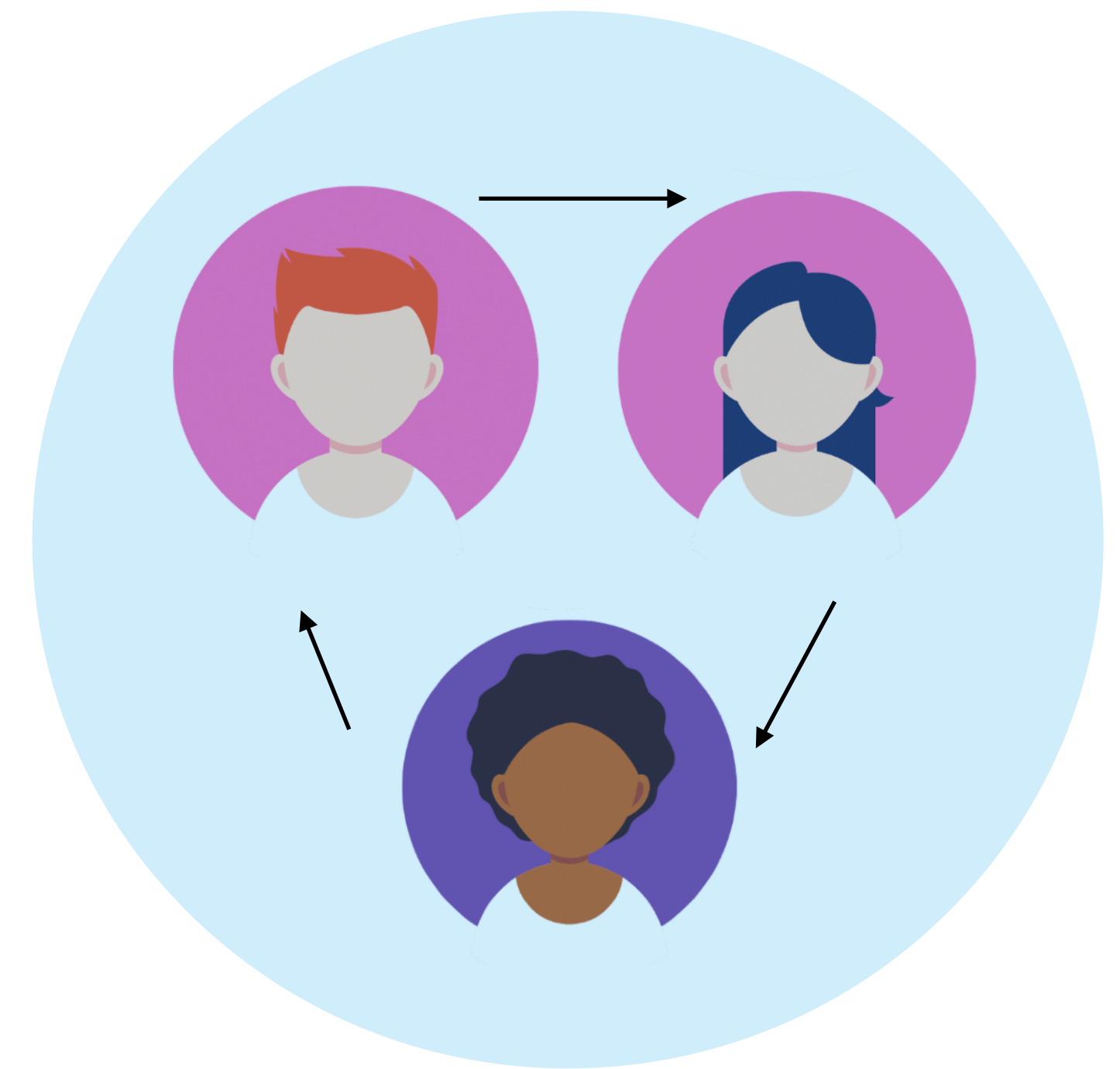
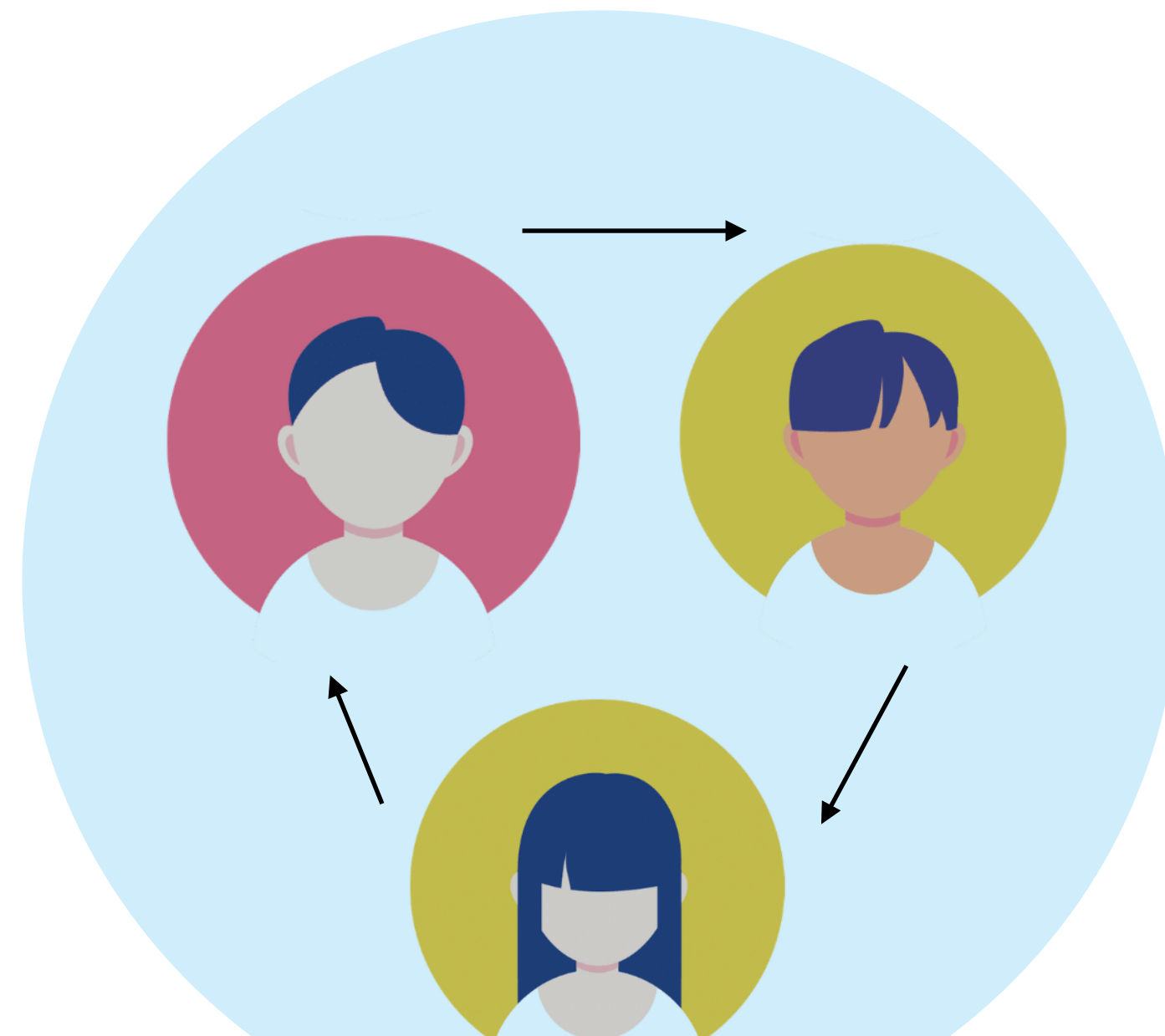
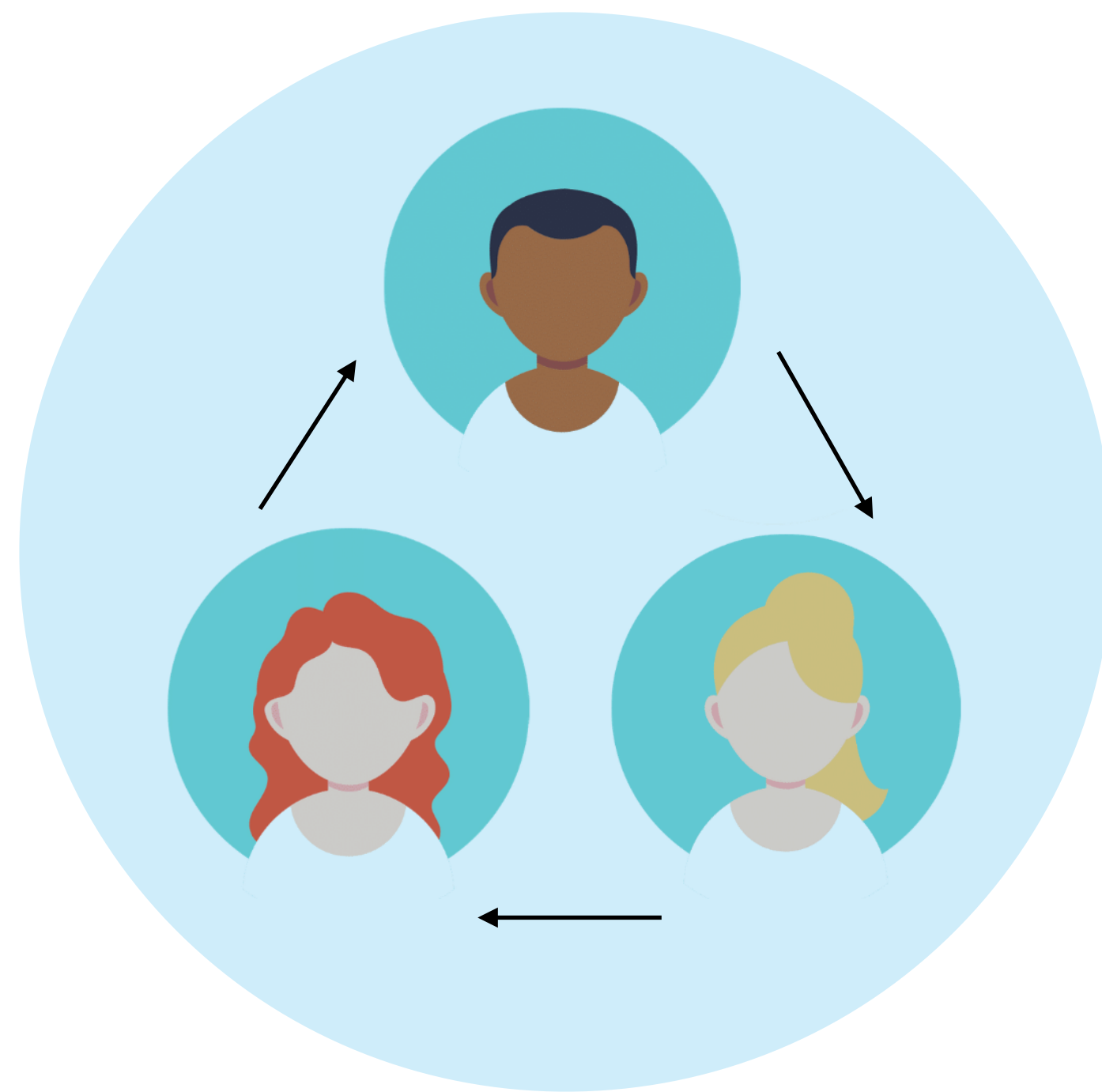
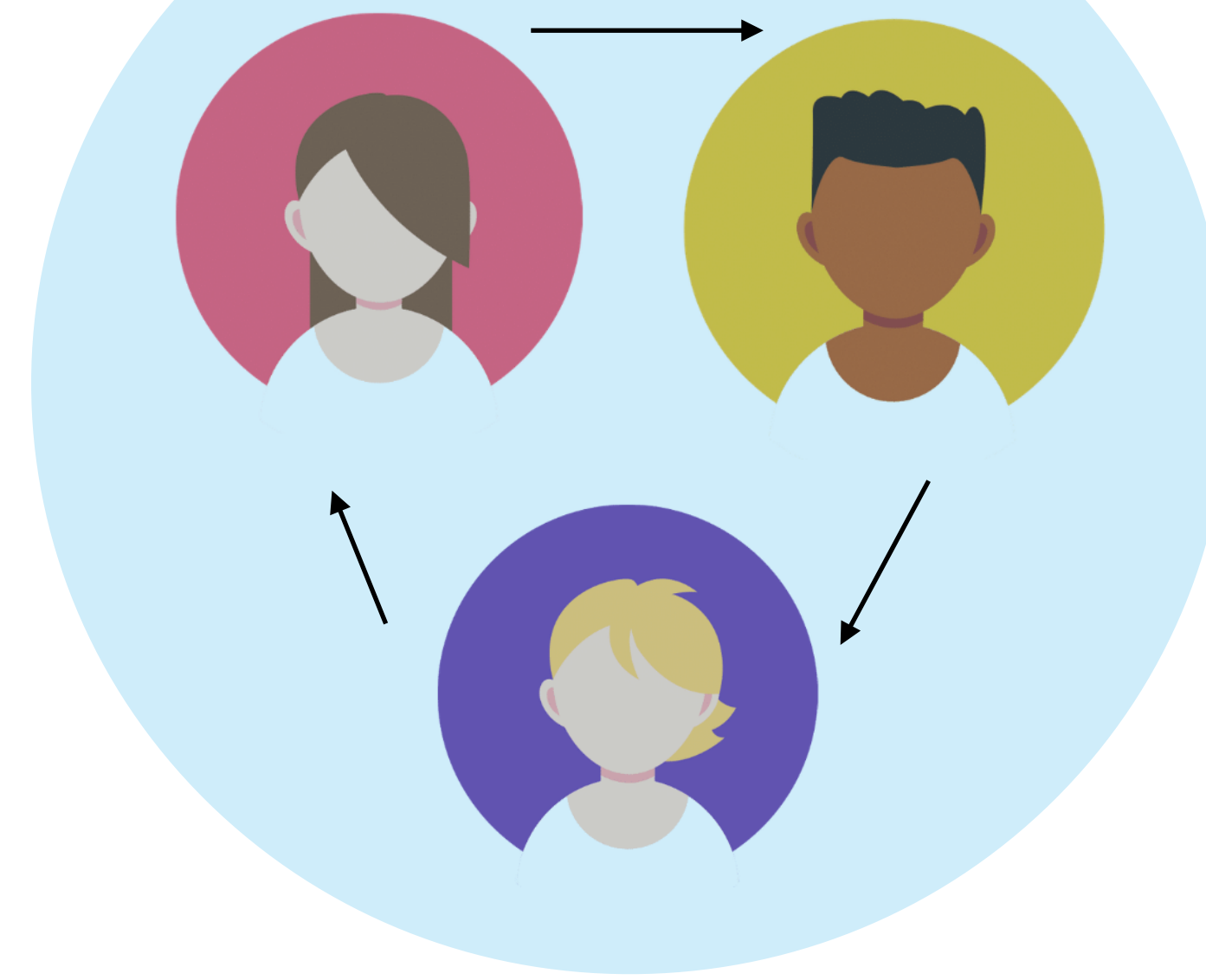
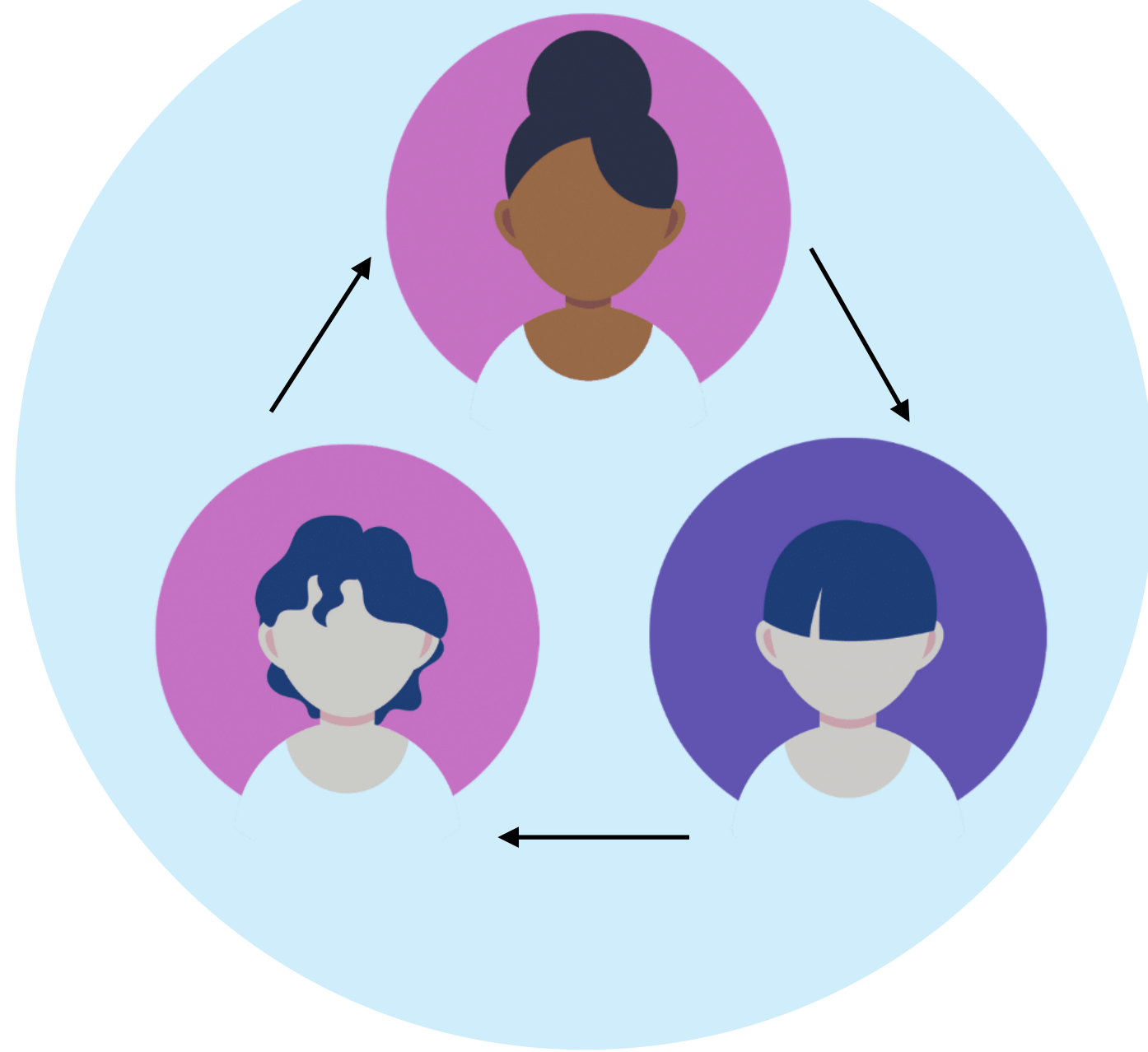


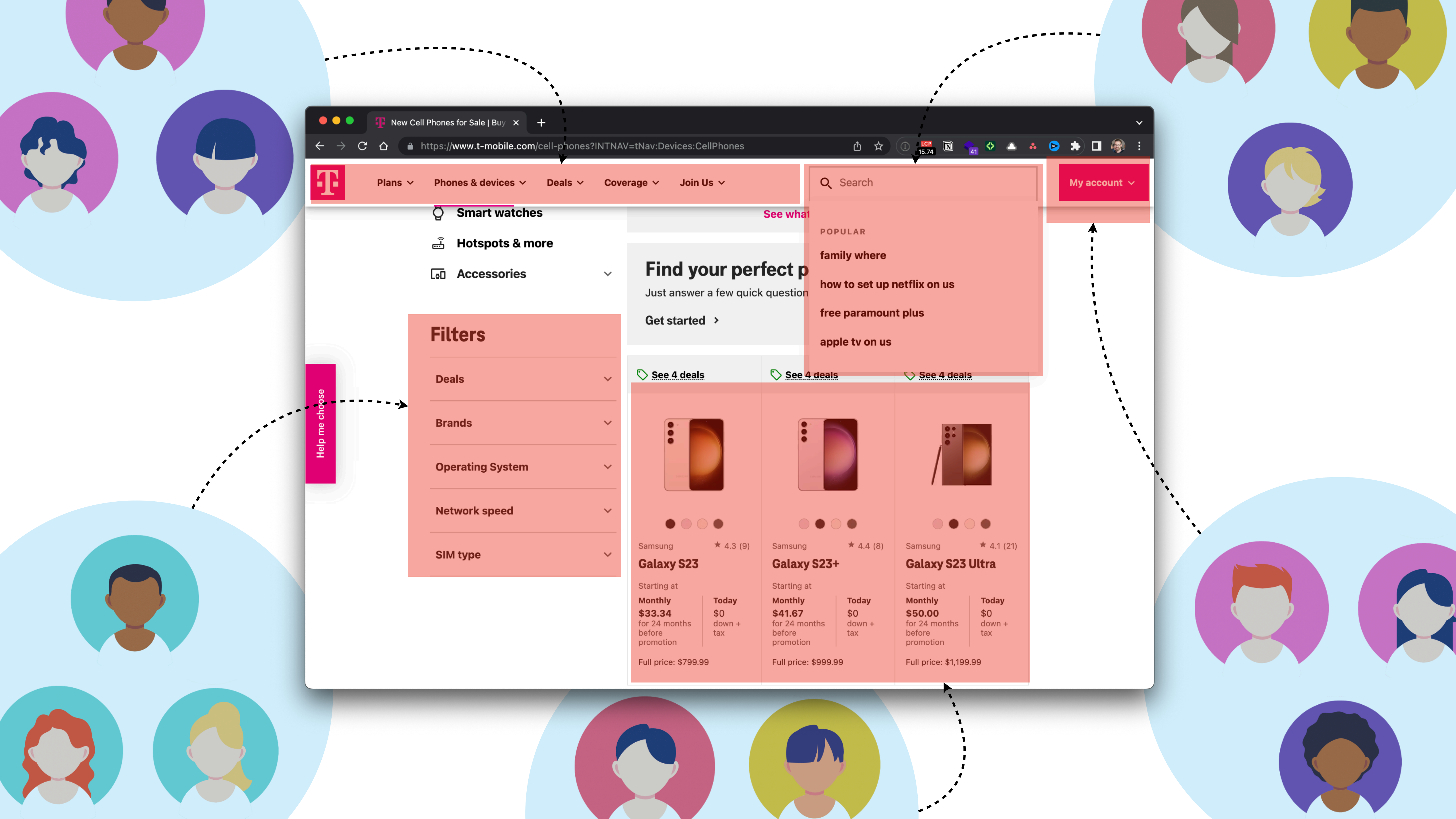
Communication Cost

Team Size

Communication Overhead







New Cell Phones for Sale | Buy

https://www.t-mobile.com/cell-phones?INTNAV=tNav:Devices:CellPhones

T

Plans

Phones & devices

Deals

Coverage

Join Us

Smart watches

Hotspots & more

Accessories

Search

My account

Find your perfect phone

Just answer a few quick questions

Get started

See 4 deals

See 4 deals

See 4 deals

Help me choose

Filters

Deals

Brands

Operating System

Network speed

SIM type

Samsung

★ 4.3 (9)

Galaxy S23

Starting at

Monthly \$33.34 for 24 months before promotion

Today \$0 down + tax

Full price: \$799.99

Samsung

★ 4.4 (8)

Galaxy S23+

Starting at

Monthly \$41.67 for 24 months before promotion

Today \$0 down + tax

Full price: \$999.99

Samsung

★ 4.1 (21)

Galaxy S23 Ultra

Starting at

Monthly \$50.00 for 24 months before promotion

Today \$0 down + tax

Full price: \$1,199.99



~~Team Size~~

of codebases

Integration Overhead Communication & Coordination

Communication Overhead



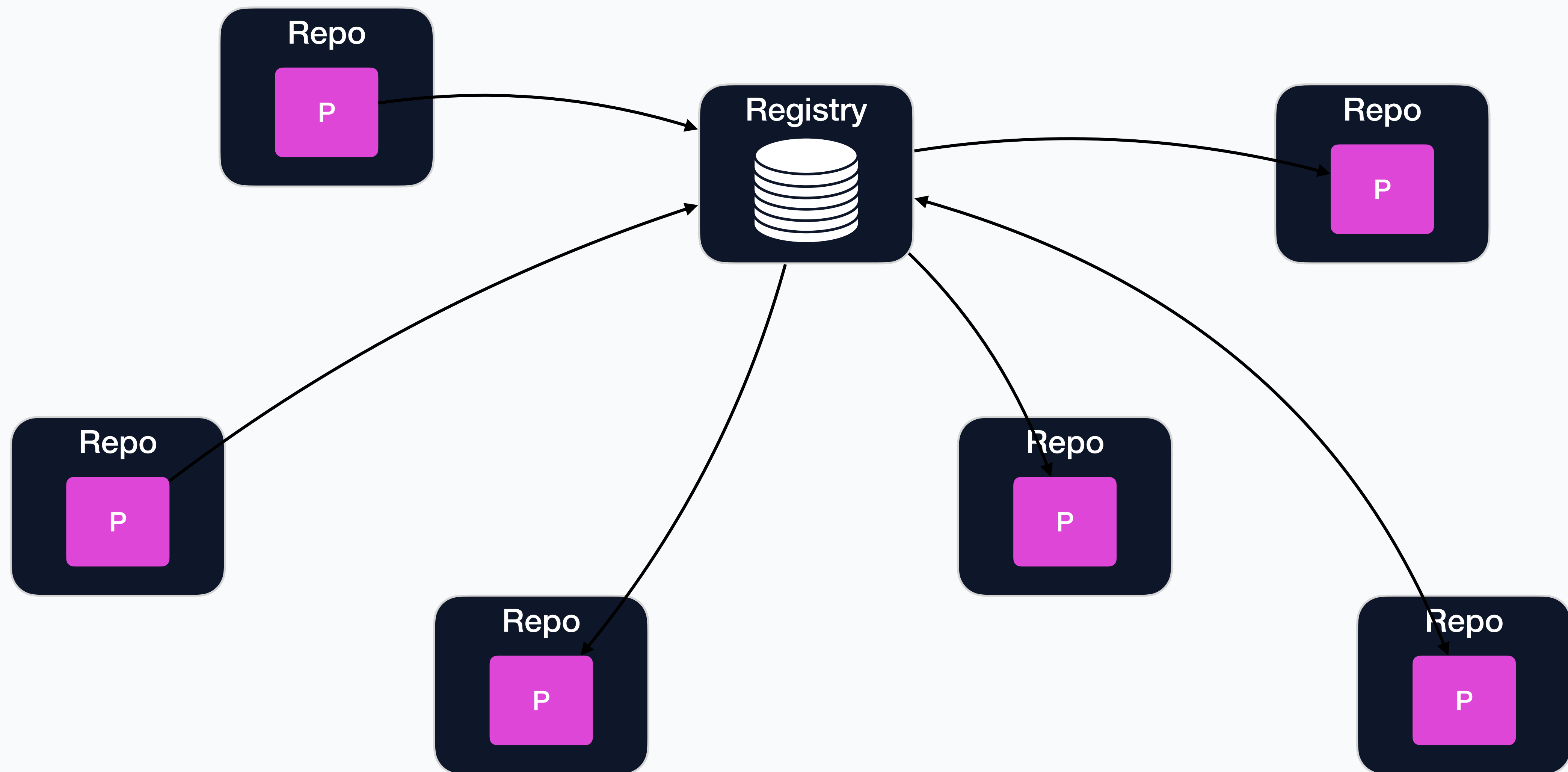
Design System Team

**“We made an update to our buttons.
Please update our design system
library to v4.3 asap.”**

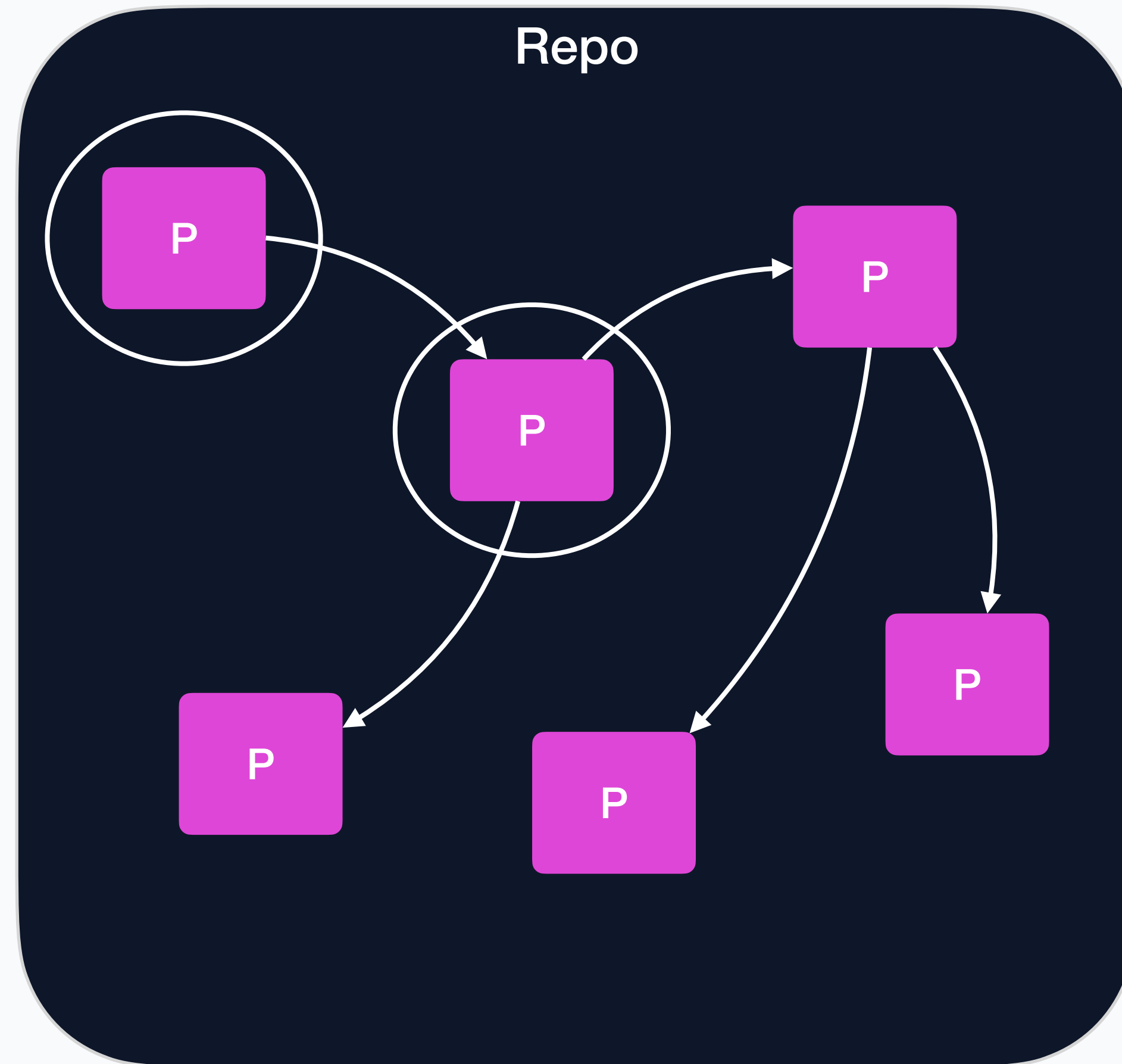
“There’s a security vulnerability in the cross-env npm package. Update all your packages that depend on it!”



Security Team



Multiple Projects per Repo





Not a silver bullet

Structure

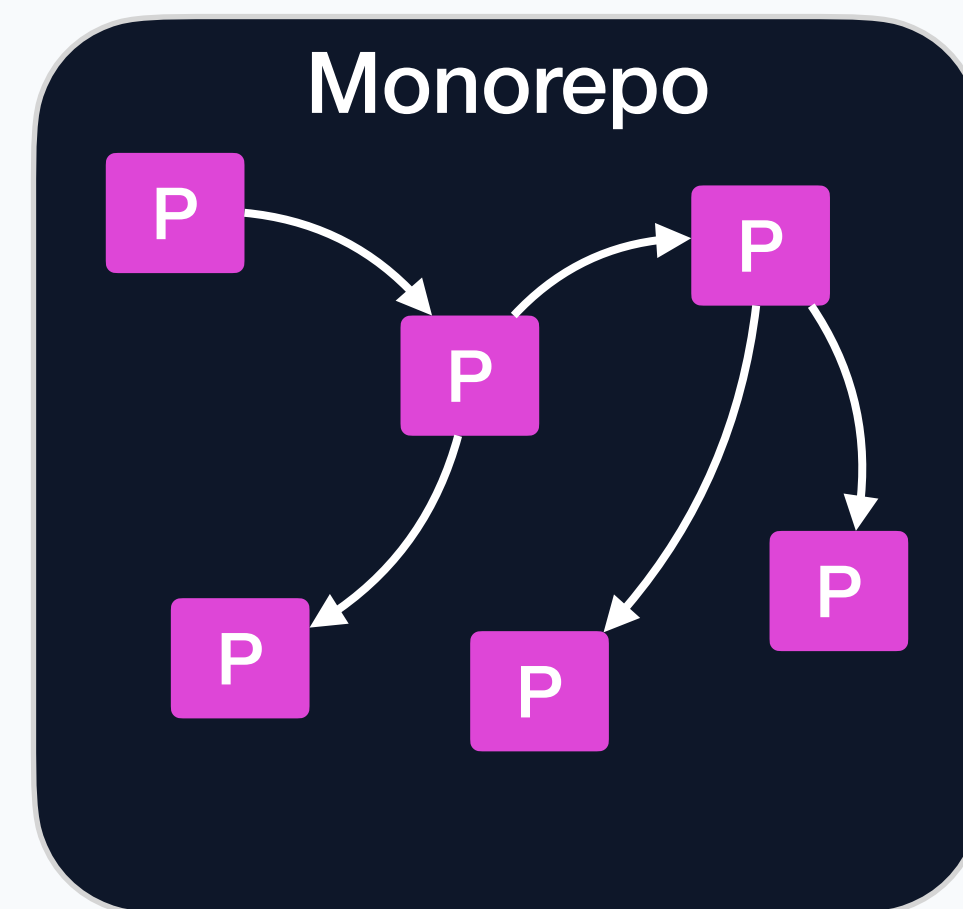
Automation

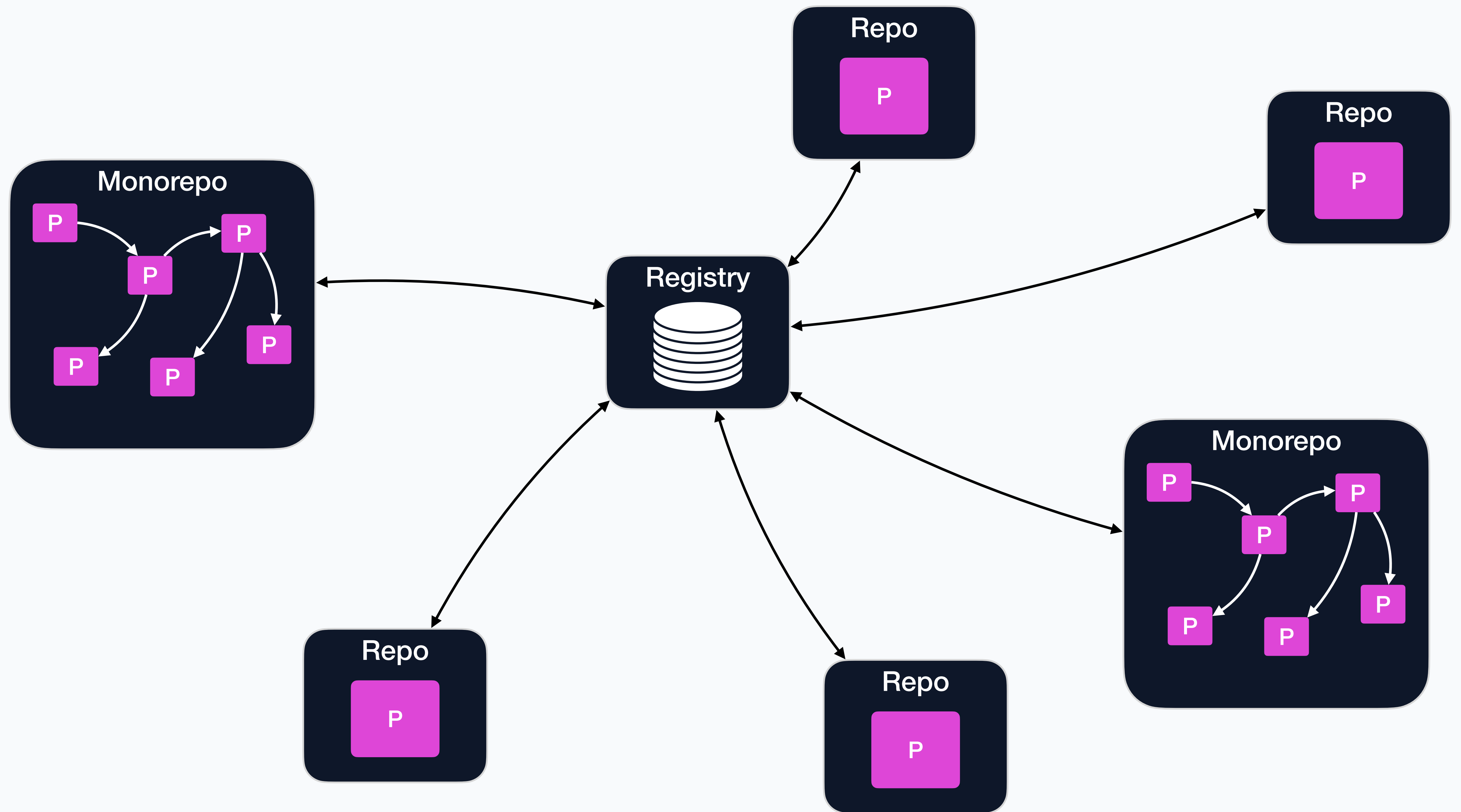
Throughput

Common Mistakes or Challenges

Common Mistake

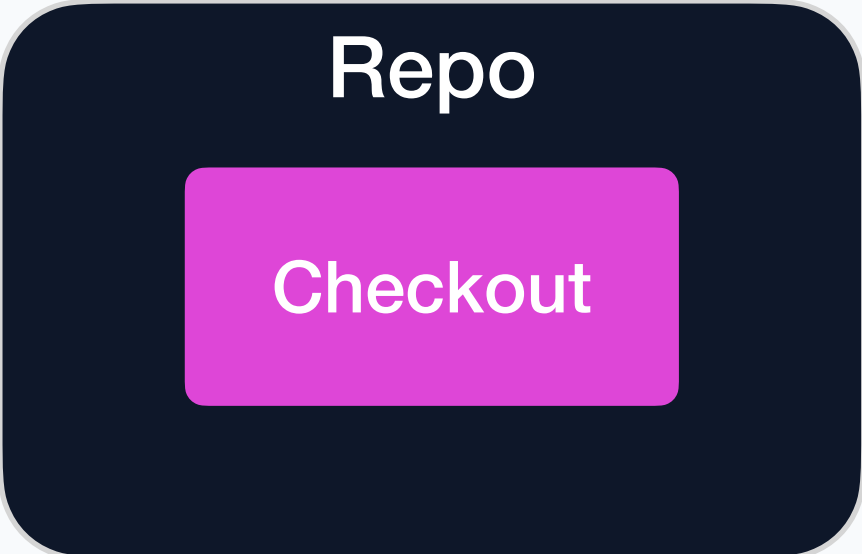
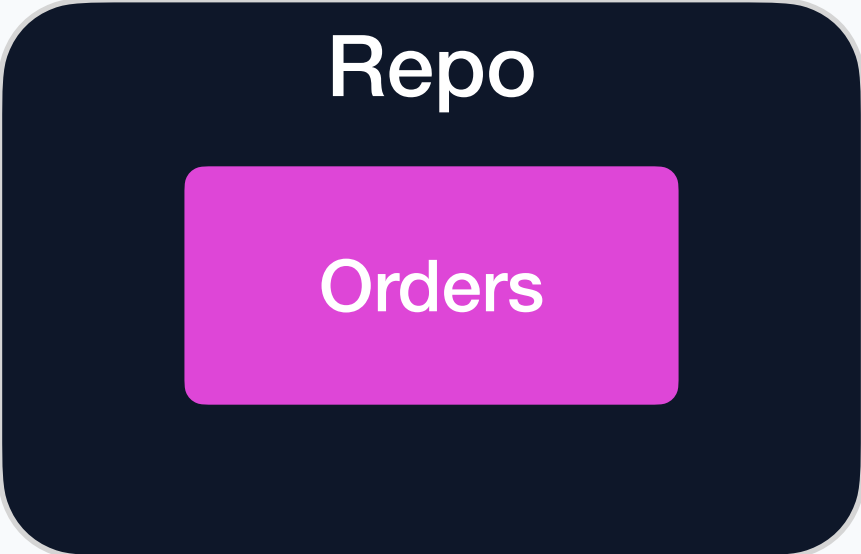
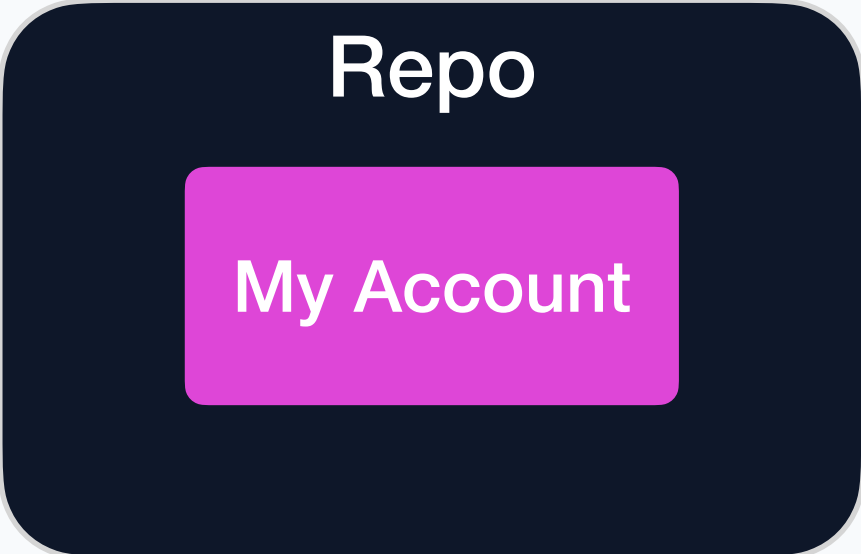
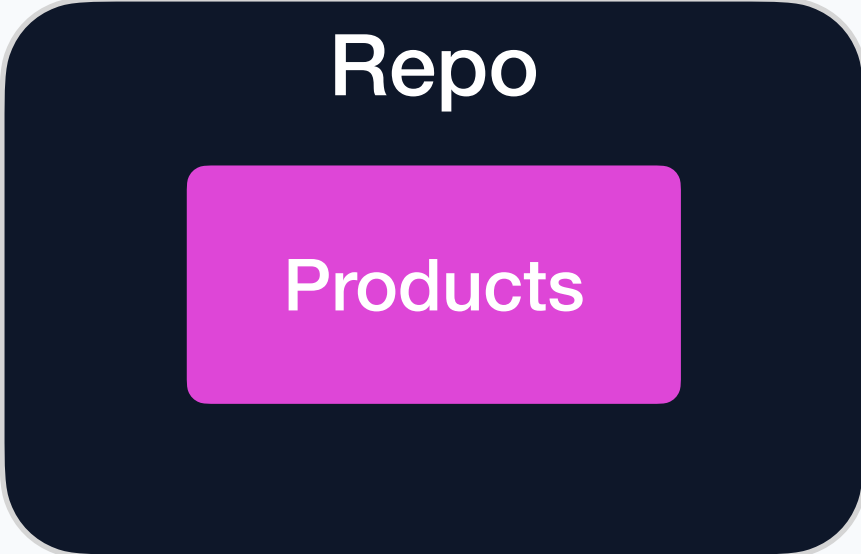
All In One

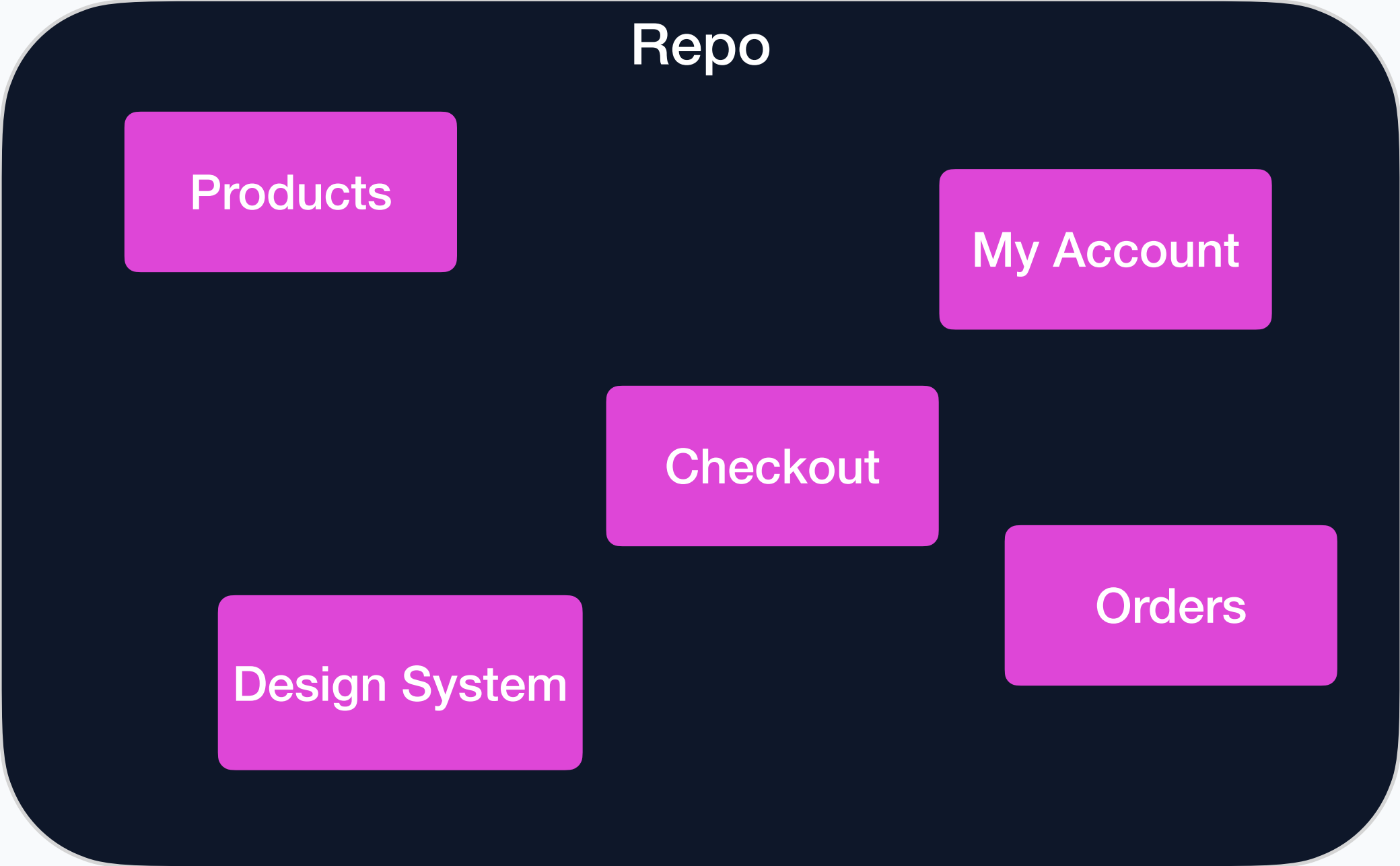


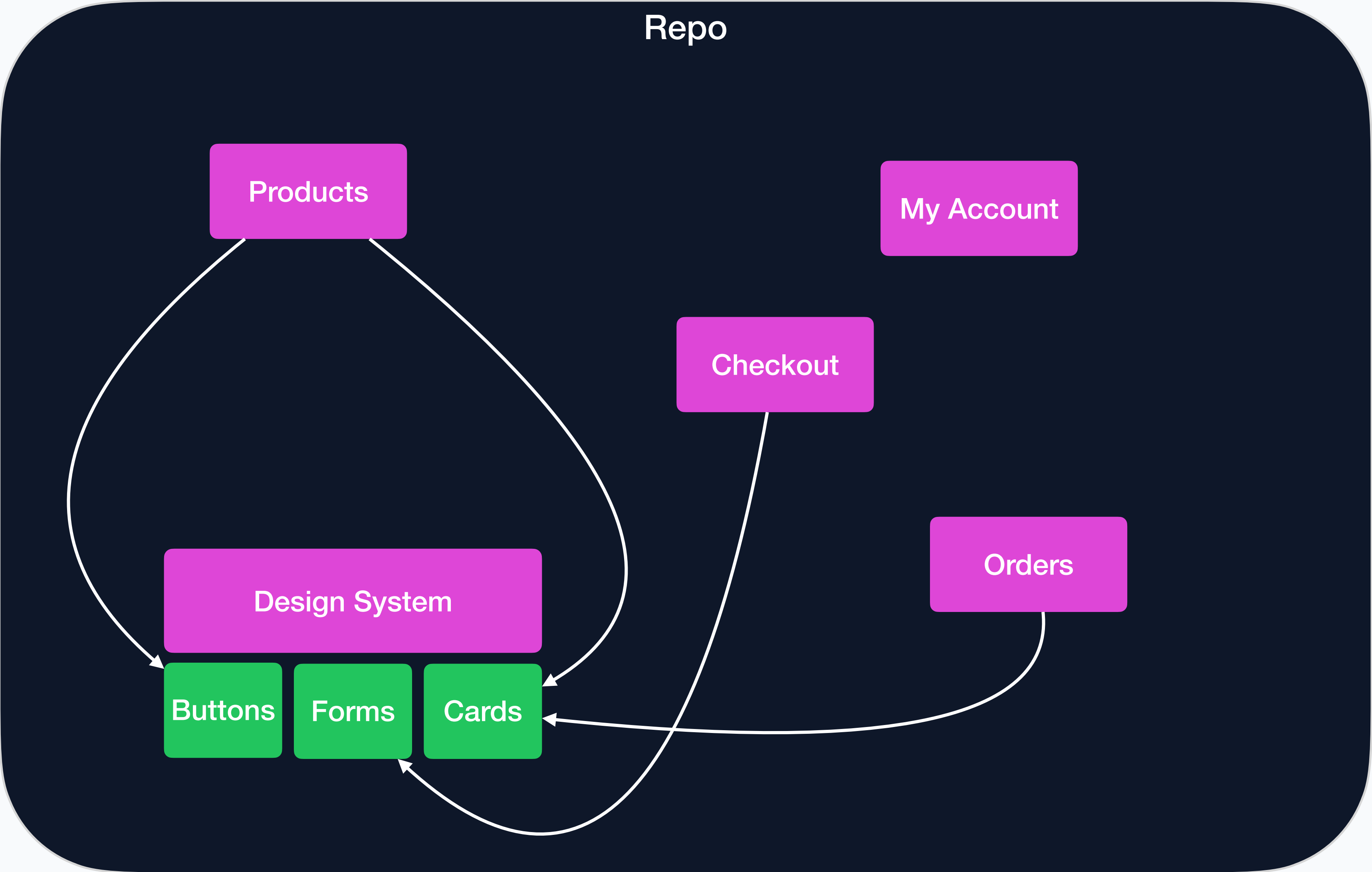


Common Mistake

**Just
Colocating
Code**







Repo

Products

My Account

Checkout

Orders

Design System

Feature

Feature

Feature

UI

UI

UI

Buttons

Forms

Cards

Domain Logic

Domain

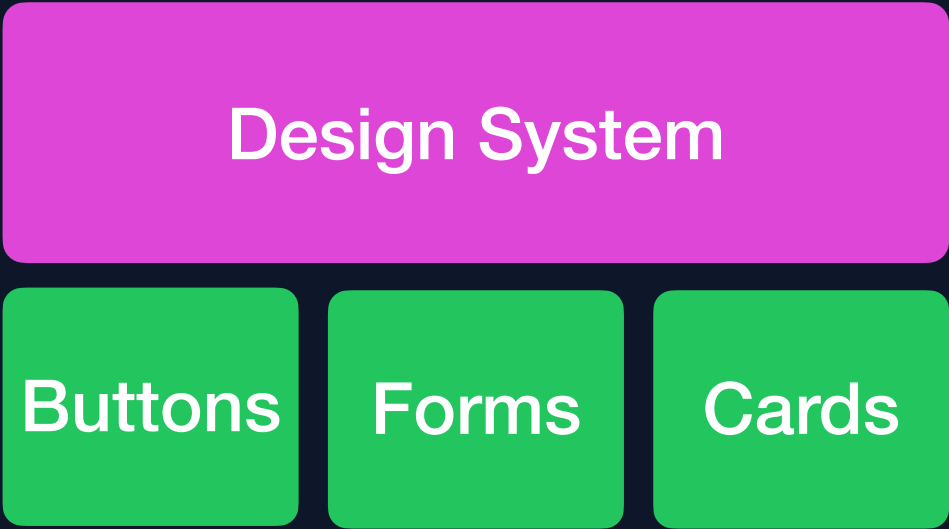
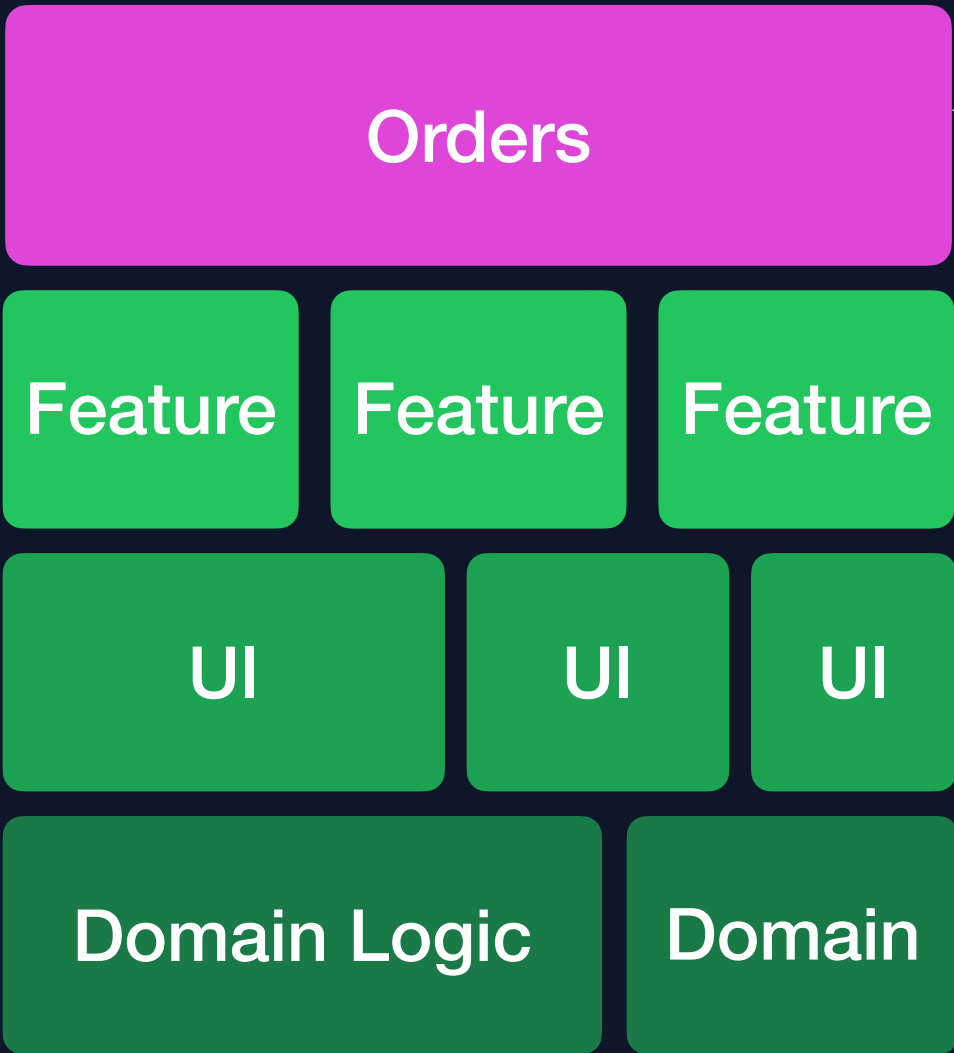
Products

Checkout

My Account

Application

Deployment container, bundling of features. Very thin; Contains top level routing to features



Products

Checkout

My Account

Orders

Feature

Feature

Feature

UI

UI

UI

Domain Logic

Domain

Specific use cases and user flows. These contain the logic of entire use cases.
Example: ordering flow

Design System

Buttons

Forms

Cards

Products

Checkout

My Account

Orders

Feature

Feature

Feature

UI

UI

UI

Domain Logic

Domain

Domain specific (re-usable) UI components

Design System

Buttons

Forms

Cards

Products

Checkout

My Account

Orders

Feature

Feature

Feature

UI

UI

UI

Domain Logic

Domain

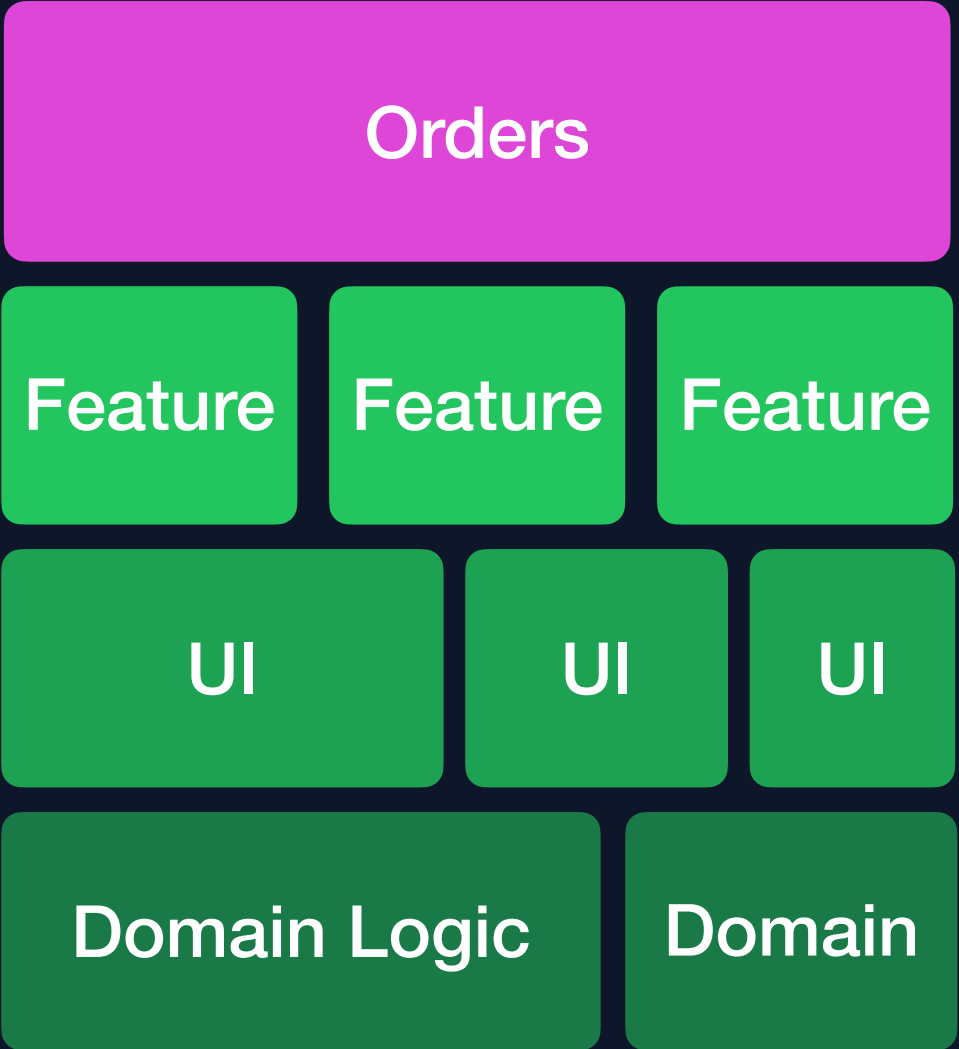
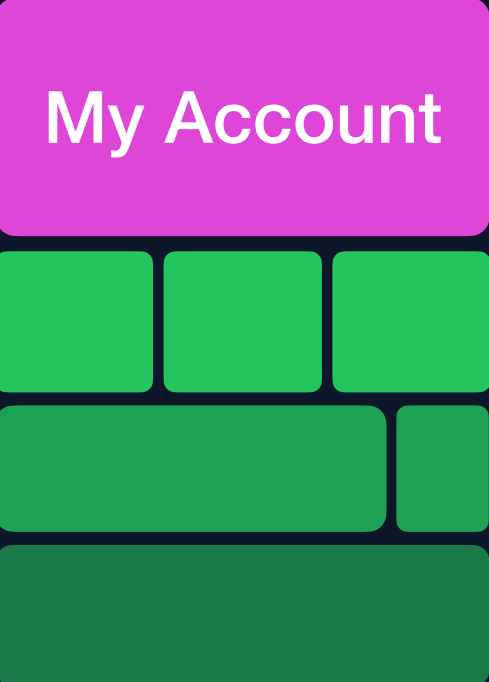
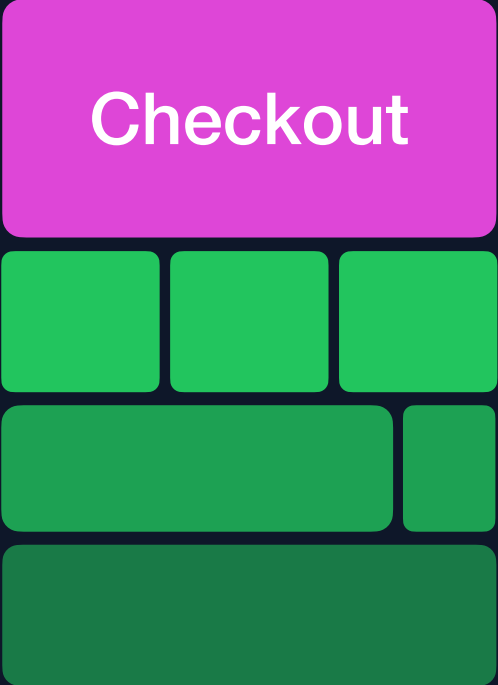
Domain logic, such as data entities, data fetching logic,...

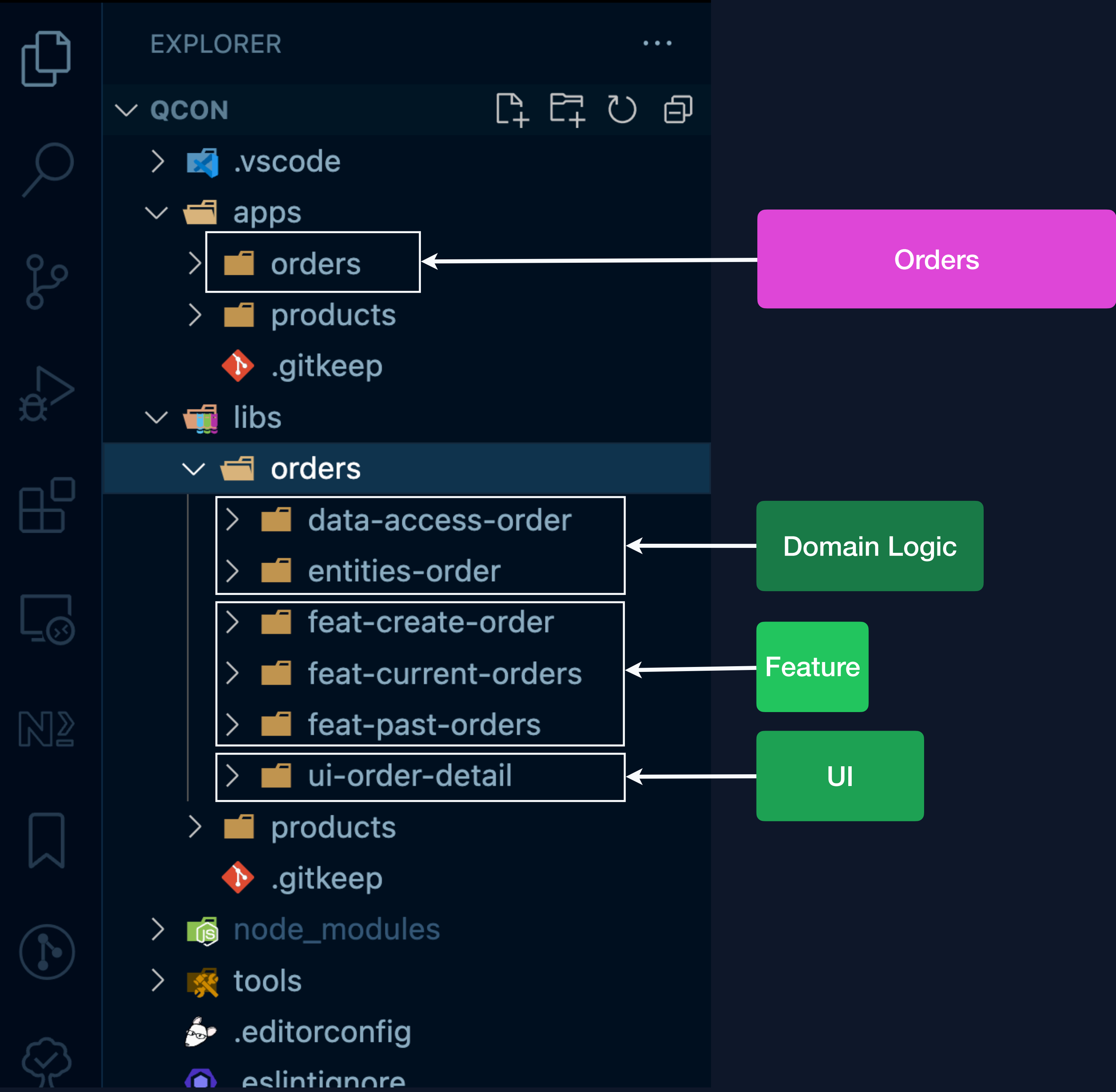
Design System

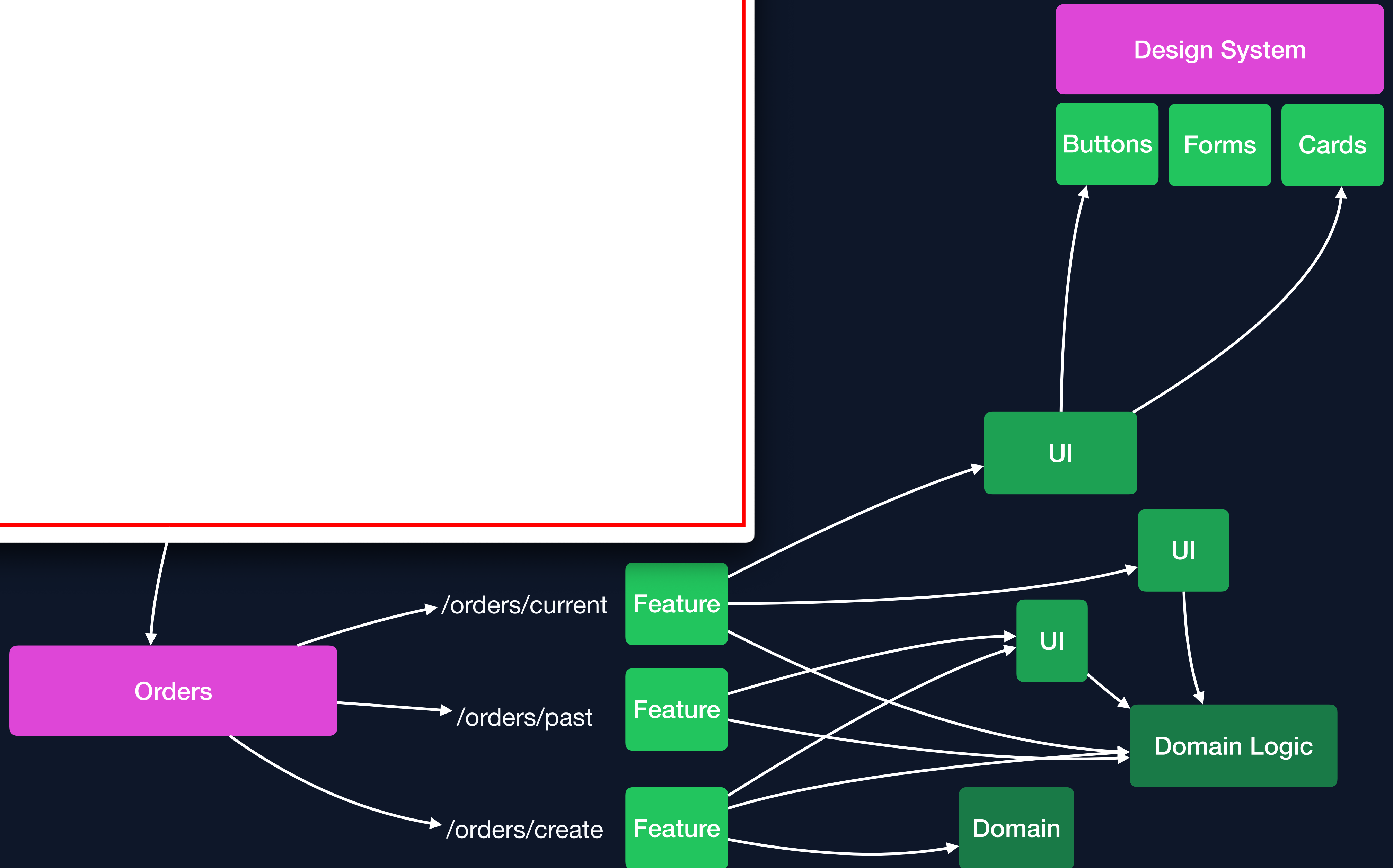
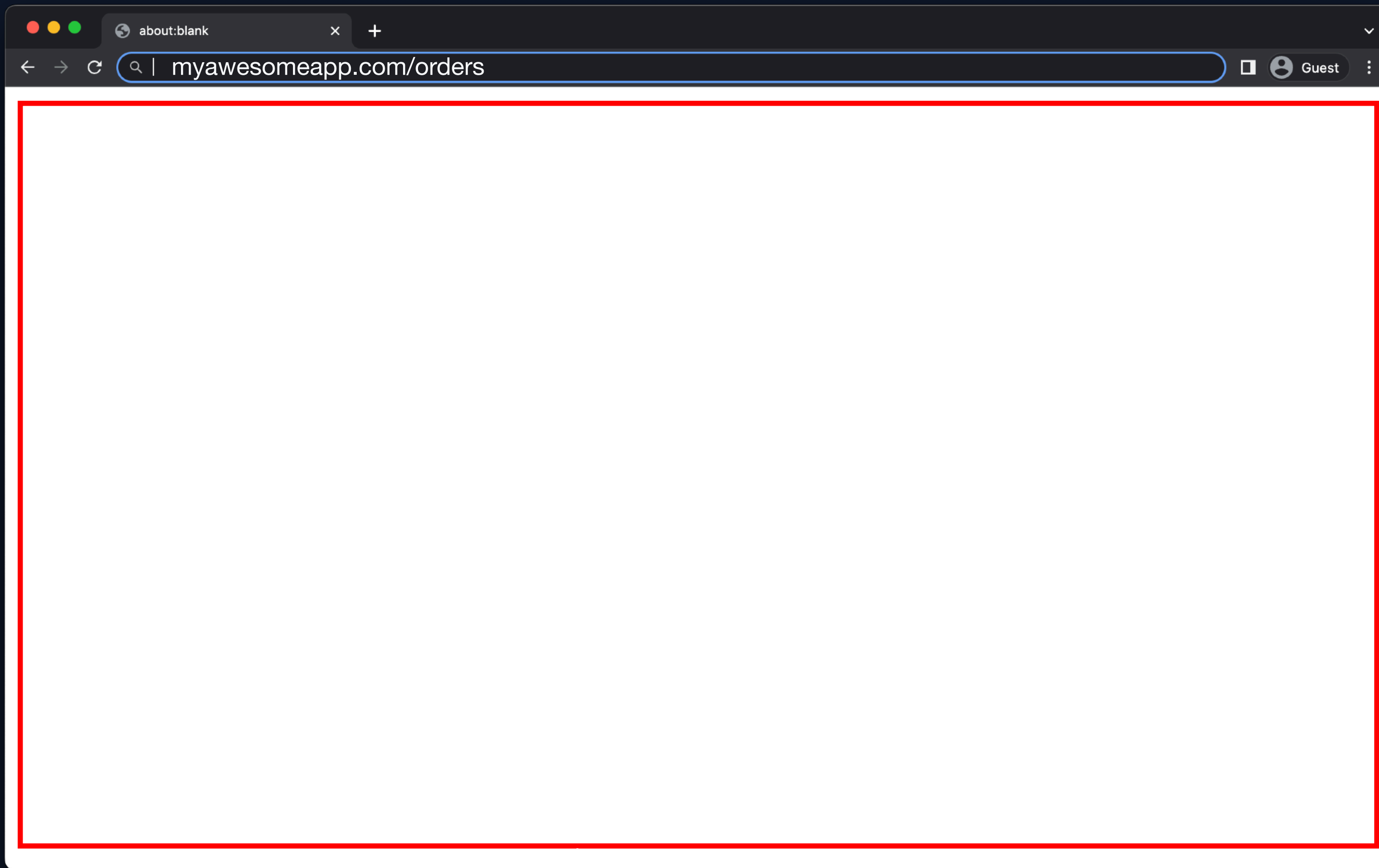
Buttons

Forms

Cards



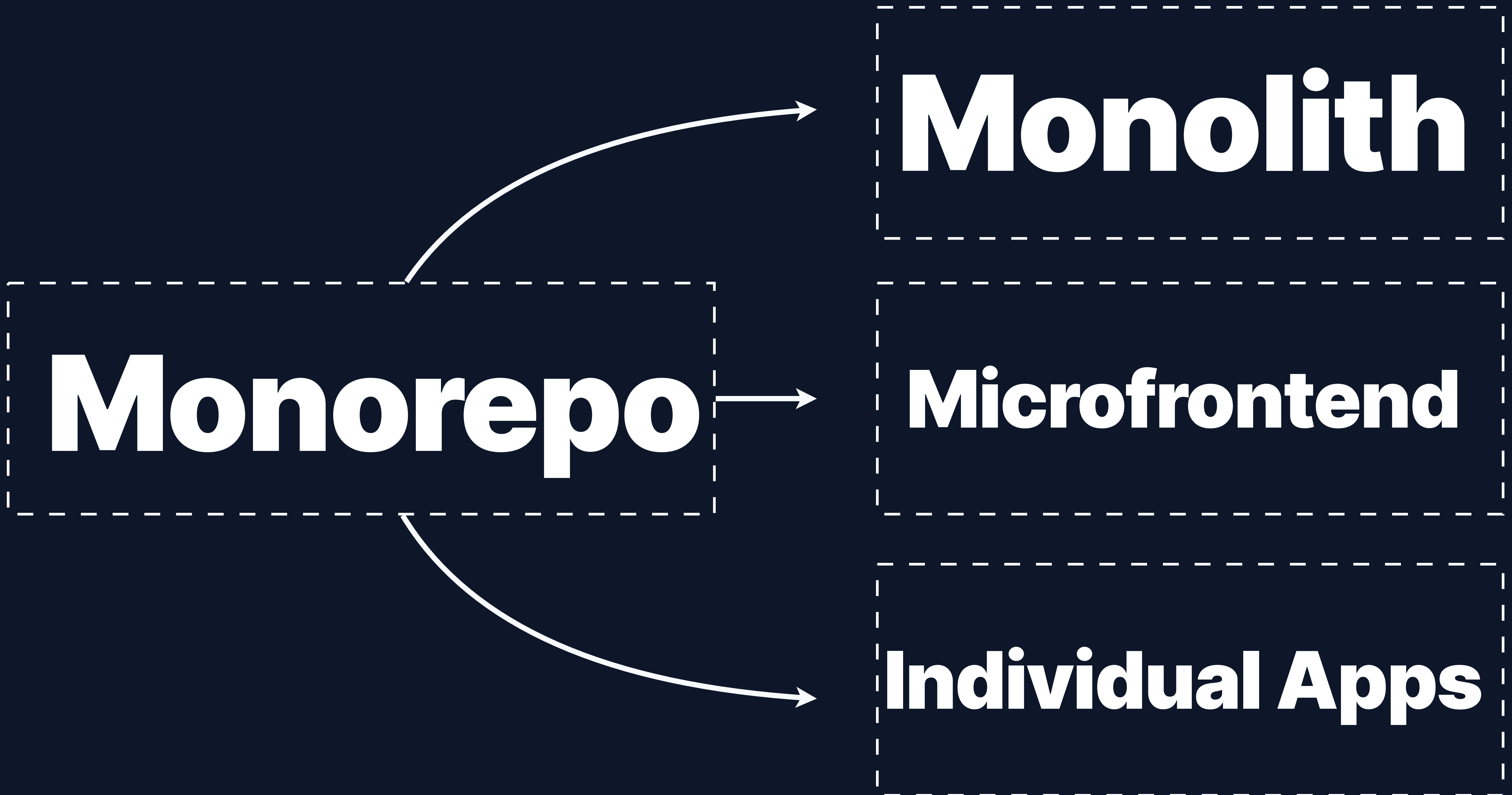




Monorepo != **Monolith**

How you **develop** your
software

How you **deploy** your
software

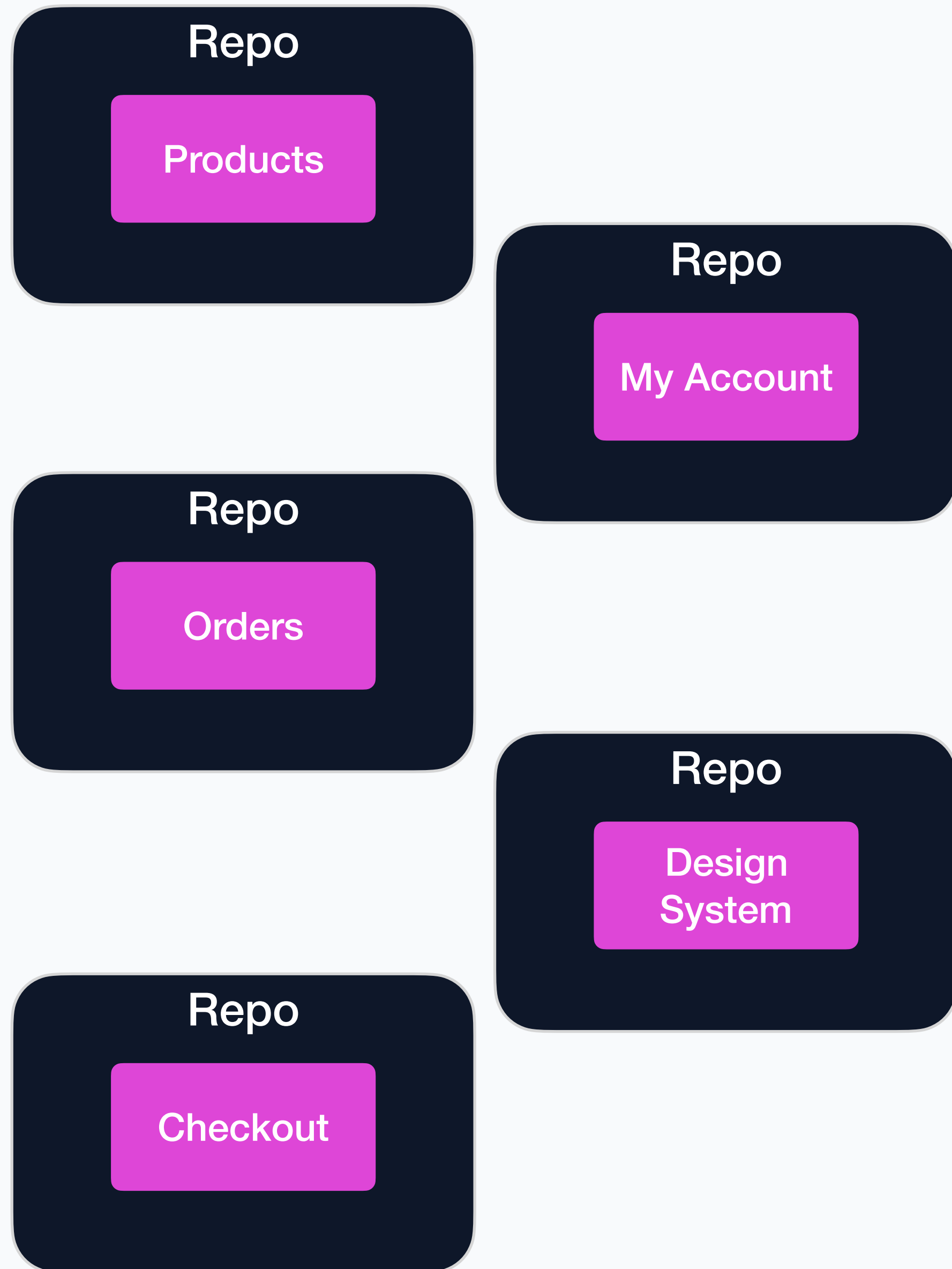


Modularity is key

- **Code Organisation & Structure**
- **Reusability**
- **Maintainability (easier to switch out features)**
- **Scaling (we'll come to that in a bit)**

Common Mistake

**Ignore
Service
Boundaries**



**“Physical” boundaries
via separate repositories**

Monorepo

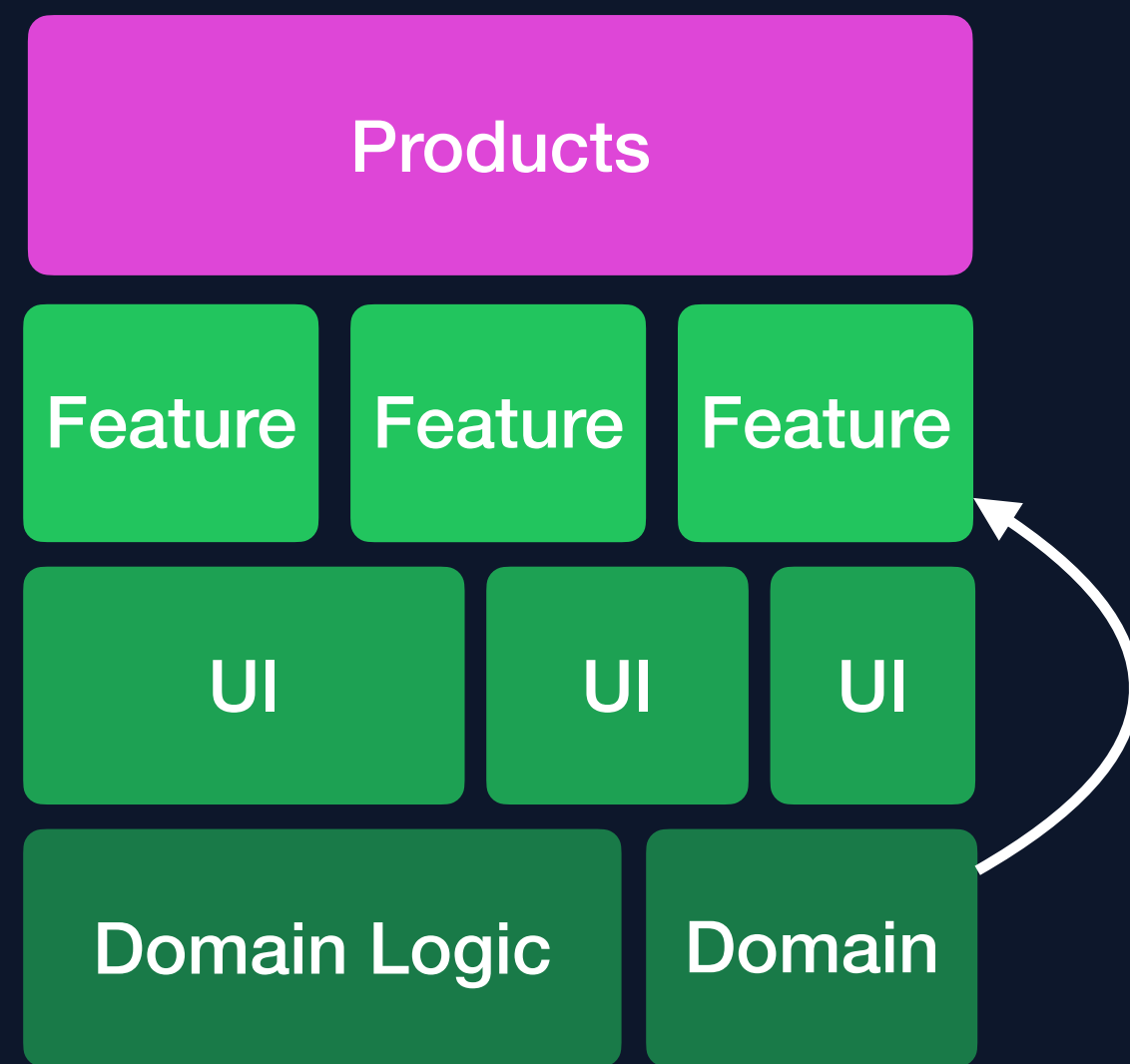
Products

Orders

My Account

Design
System

Checkout



Control dependencies
within a domain area

Across Domain Areas

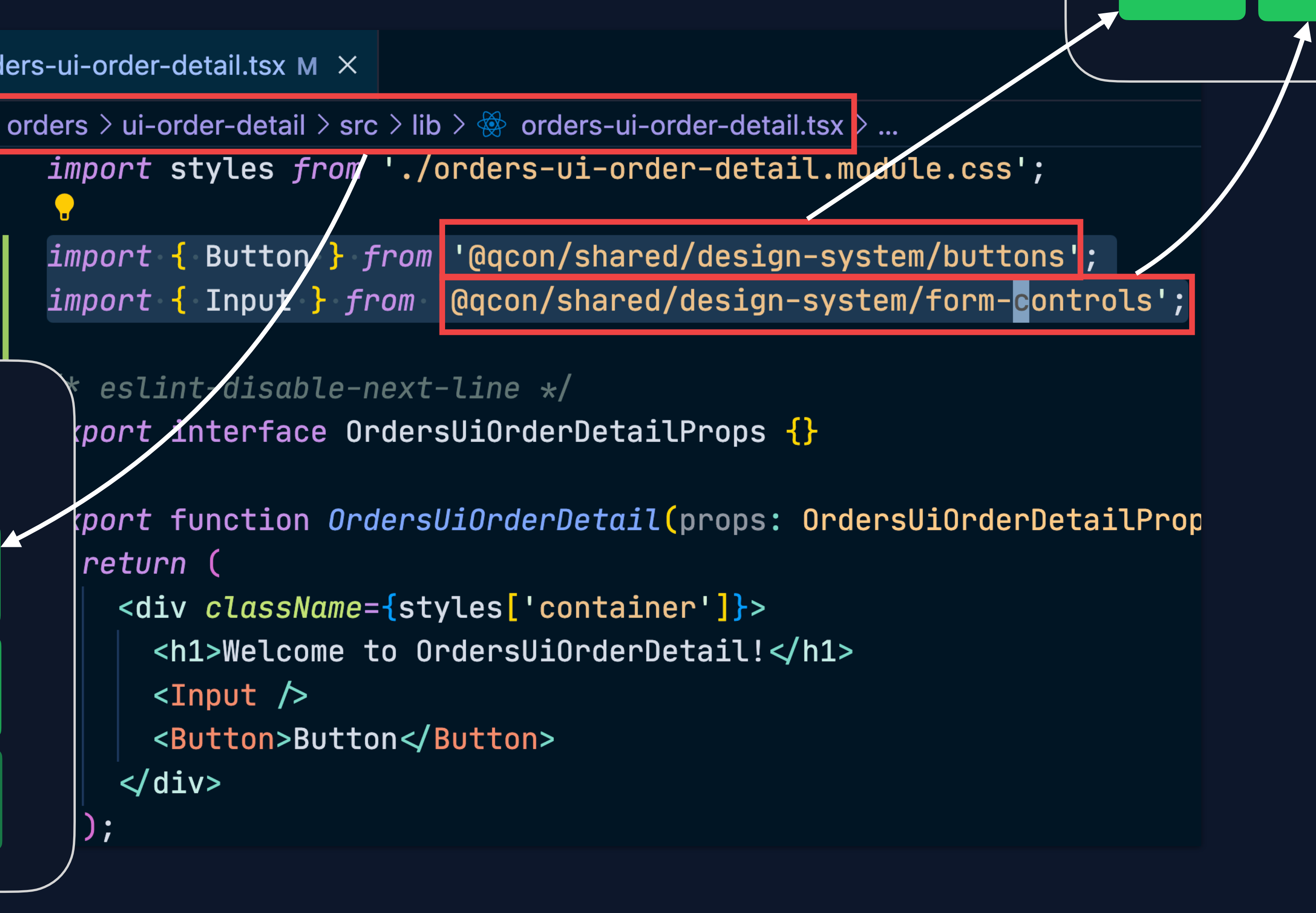
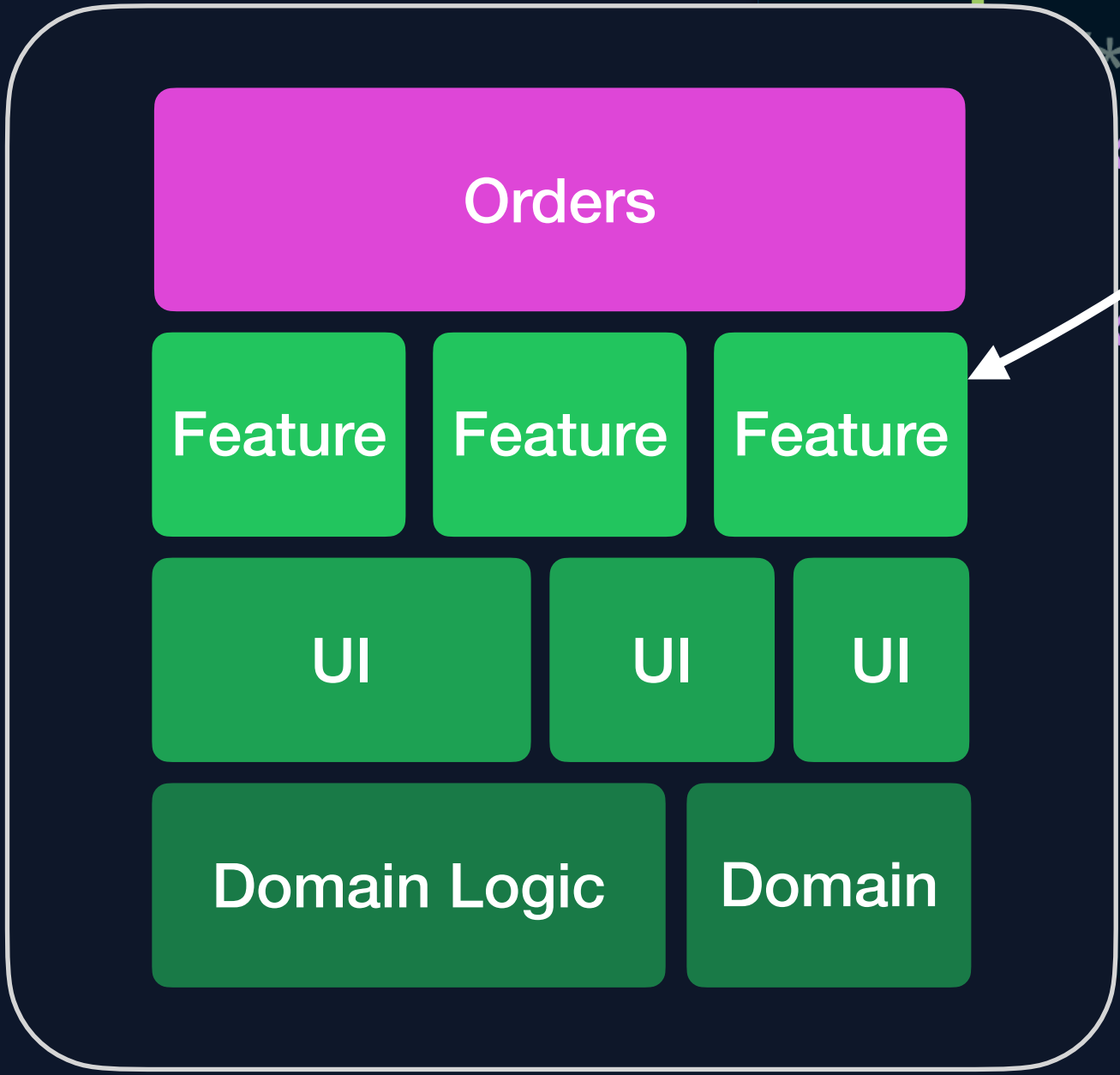
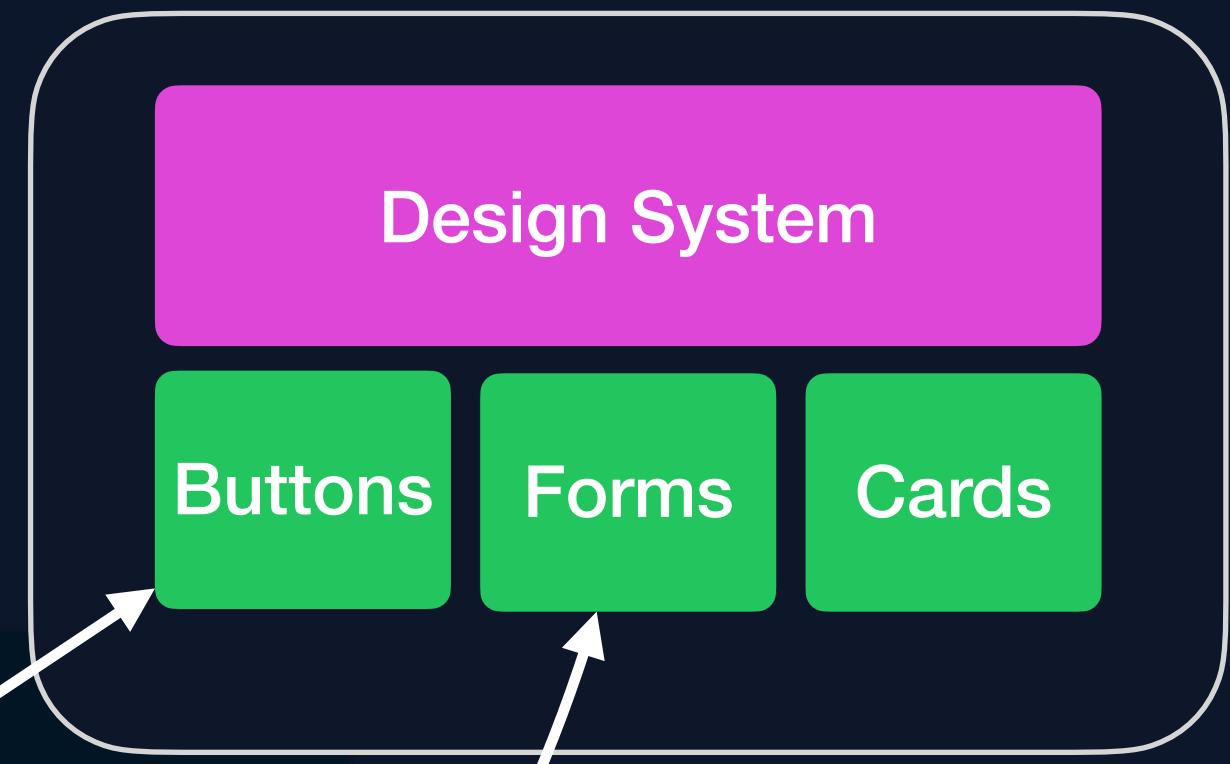


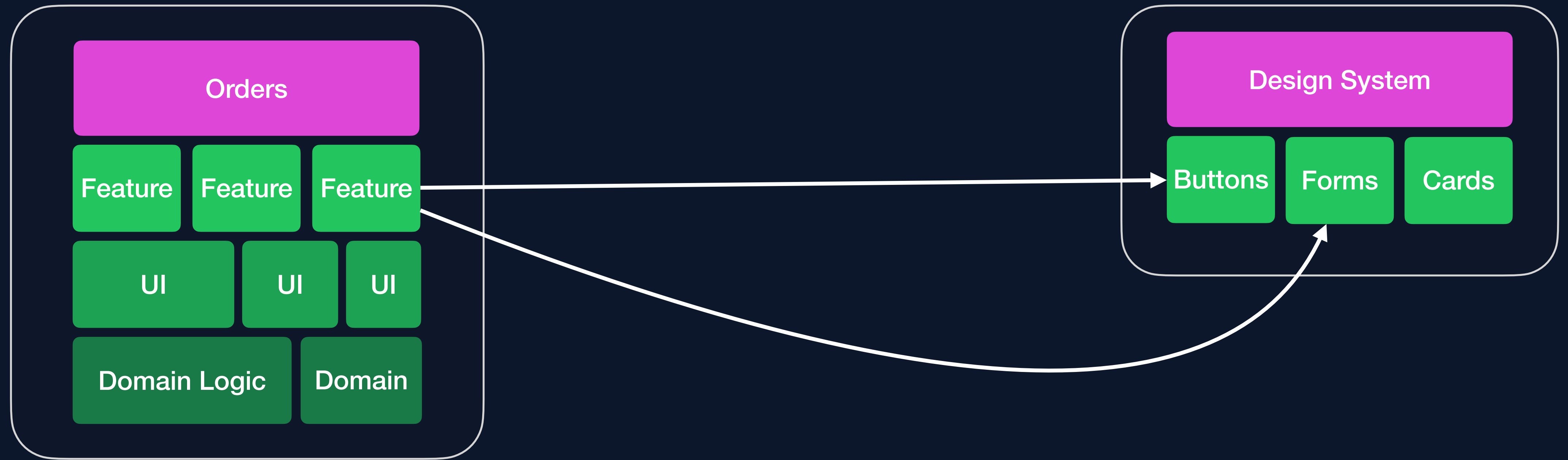
orders-ui-order-detail.tsx M X

libs > orders > ui-order-detail > src > lib > orders-ui-order-detail.tsx > ...

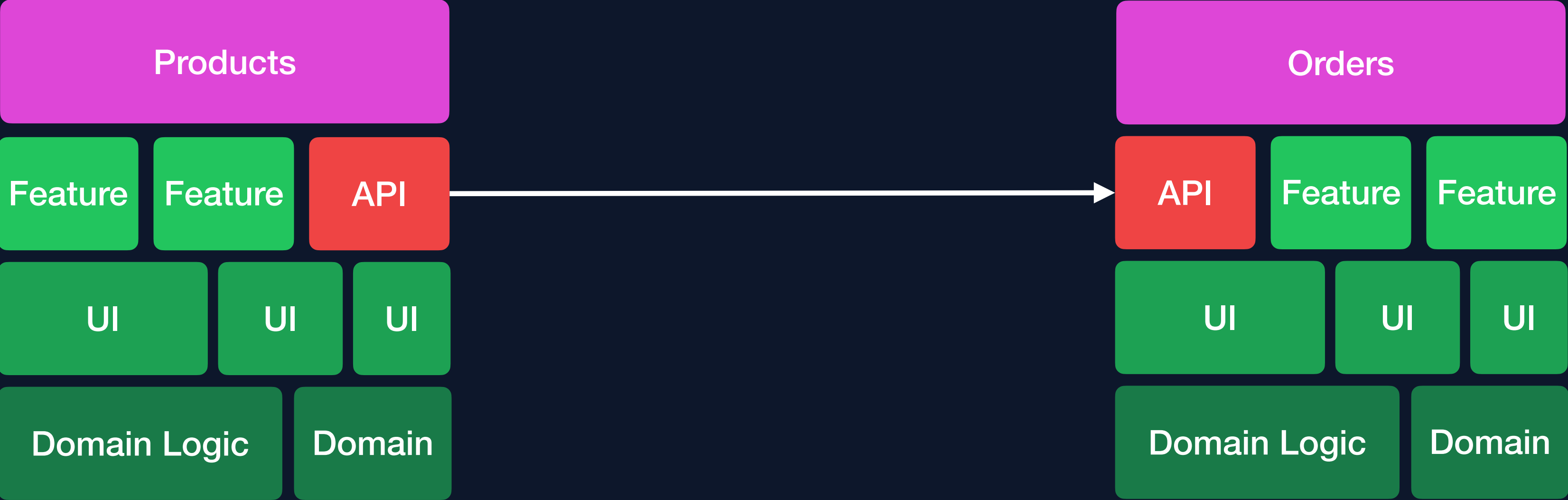
```
3 import styles from './orders-ui-order-detail.module.css';
2
1 import { Button } from '@qcon/shared/design-system/buttons';
4 import { Input } from '@qcon/shared/design-system/form-controls';
1
/* eslint-disable-next-line */
export interface OrdersUiOrderDetailProps {}

export function OrdersUiOrderDetail(props: OrdersUiOrderDetailProp
return (
  <div className={styles['container']}>
    <h1>Welcome to OrdersUiOrderDetail!</h1>
    <Input />
    <Button>Button</Button>
  </div>
);
```





1. What **type of project** can depend on which other **type**?
2. Which **scope** (or **domain**) can depend on what other scope?



Type

Feature

UI

Domain
Logic

Scope

Orders

My
Account

Products

Checkout

Shared
(Design System, Utils)



{ } project.json M ✕

libs > orders > ui-order-detail > { } project.json > ...

```
5  {
4    "name": "orders-ui-order-detail",
3    "$schema": "../../../node_modules/nx/schemas/project-schema.json",
2    "sourceRoot": "libs/orders/ui-order-detail/src",
1    "projectType": "library",
6  "tags": ["scope:orders", "type:ui"],
1  "targets": {
    nx run orders-ui-order-detail:lint
2    "lint": {
3      "executor": "@nrwl/linter:eslint",
4      "outputs": ["{options.outputFile}"]
```

.eslintrc.json > [] overrides > {} 0 > {} rules > [] @nrwl/nx/enforce-module-boundaries > {} 1 > [] depConstraints

```
26 "depConstraints": [  
25   {  
24     "sourceTag": "*",  
23     "onlyDependOnLibsWithTags": ["*"]  
22   },  
21   {  
20     "sourceTag": "type:feature",  
19     "onlyDependOnLibsWithTags": [  
18       "type:feature",  
17       "type:ui",  
16       "type:util",  
15       "type:domain"  
14     ]  
13   },  
12   {  
11     "sourceTag": "type:domain",  
10     "onlyDependOnLibsWithTags": ["type:util"]  
9   },  
8   {  
7     "sourceTag": "type:ui",  
6     "onlyDependOnLibsWithTags": ["type:ui", "type:util"]  
5   },  
4   {  
3     "sourceTag": "type:util",  
2     "onlyDependOnLibsWithTags": ["type:util"]  
1   }  
]
```

Type → Type

```

41 "depConstraints": [
40   {
39     "sourceTag": "*",
38     "onlyDependOnLibsWithTags": ["*"]
37   },
36   { ...
28   },
27   { ...
24   },
23   { ...
20   },
19   { ...
16   },
15   {
14     "sourceTag": "scope:products",
13     "onlyDependOnLibsWithTags": ["scope:products", "scope:shared"]
12   },
11   {
10     "sourceTag": "scope:orders",
9     "onlyDependOnLibsWithTags": ["scope:orders", "scope:shared"]
8   },
7   {
6     "sourceTag": "scope:myaccount",
5     "onlyDependOnLibsWithTags": ["scope:myaccount", "scope:shared"]
4   }
3 ]
2 }
1 ]

```

Domain $\rightarrow$ Domain

Enforcing Boundaries

- **Avoids spaghetti dependencies**
- **Keeps our codebase healthy as it grows**
- **Maintainability (!!)**
- **Needs to be enforced automatically**

Common Mistake

**Not investing in
Automation**

Key to adoption:

Adding a new project to your mono repo should be easy.

At a certain point, doing operations
manually won't scale.

libs > products > feat-product-list > src > lib > products-feat-product-list.tsx > ...

```
1 import styles from './products-feat-product-list.module.css';
2 import { OrdersUiOrderDetail } from '@qcon/orders/ui-order-detail';
3
4 A project tagged with "scope:products" can only depend on
5 libs tagged with "scope:products",
6 "scope:shared" eslint(@nrwl/nx/enforce-module-boundaries)
7 View Problem (⌘F8) Quick Fix... (⌘.)
8
9 export function ProductsFeatProductList(props: ProductsFeatProductListProps) {
10   return (
11     <div className={styles['container']}>
12       <h1>Welcome to ProductsFeatProductList!</h1>
13       <OrdersUiOrderDetail />
14     </div>
15   );
16 }
17
18 export default ProductsFeatProductList;
```

Code Generation



```
~ . . . . .  
> npx nx g @nx/react:app myreactapp --directory=apps  
npx nx g @nx/react:lib product-list --directory=libs/products  
npx nx g @nx/react:lib product-detail --directory=libs/products  
npx nx g @nx/react:lib create-order --directory=libs/orders
```

A terminal window with a dark background and light gray text. The window has a title bar with three colored circles (red, yellow, green) on the left and a symbol on the right. The terminal content shows a series of commands for generating code with Nx. The first command is preceded by a green prompt character. The subsequent commands are indented. The terminal also shows a blue tilde character and a series of dots on the line above the first command.

Code Generation

Creates project files (READMEs,...)

Configures the build/test/lint tooling

Sets CODEOWNER files

Configures the boundary rules

Enforce company coding style guide

Incrementally add features



Bonus from generating:
Consistency



Automatic Upgrades

Code Migrations

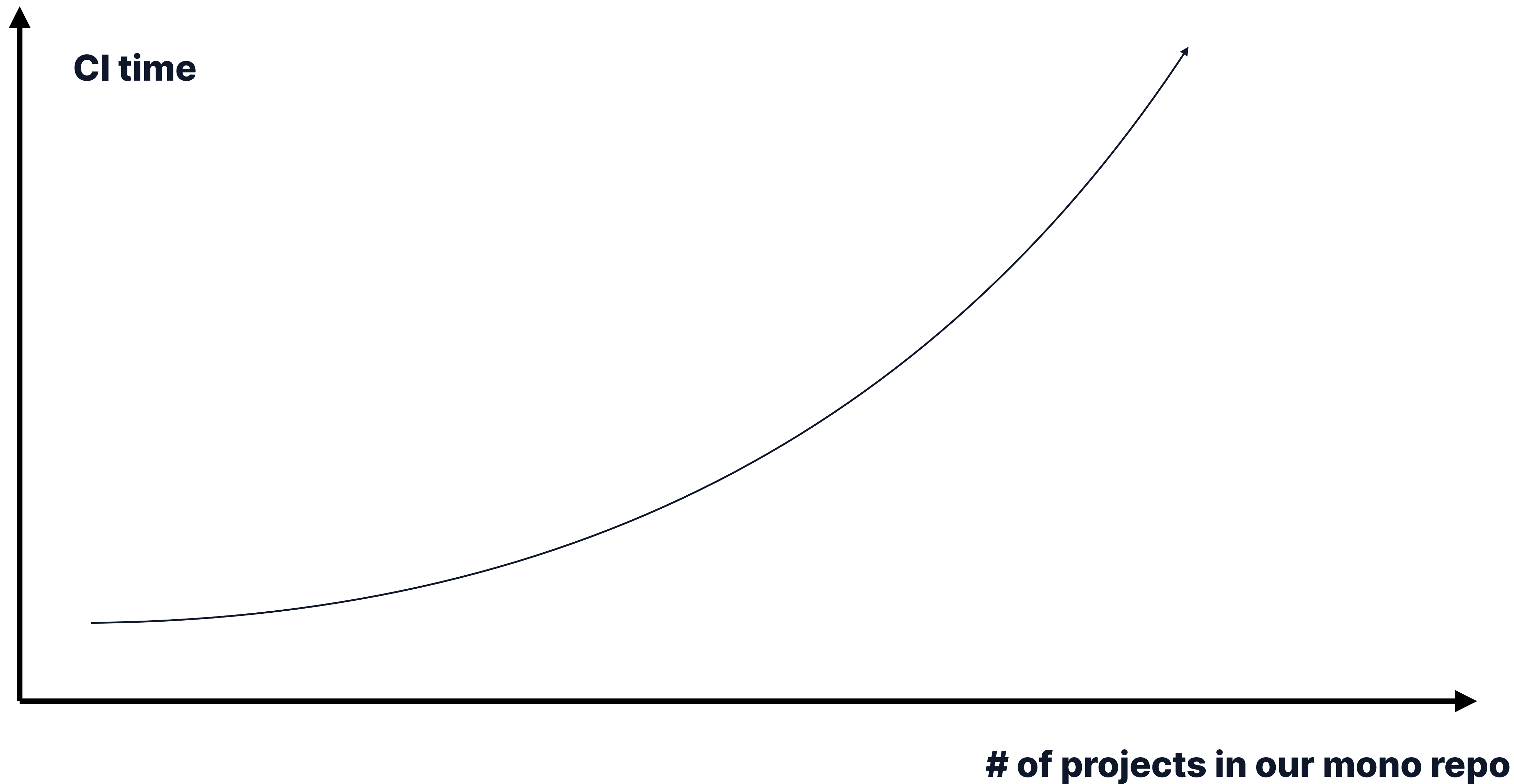


```
~  
> npx nx migrate latest
```

A terminal window with a dark background and a light gray title bar. The title bar contains three colored window control buttons (red, yellow, green) on the left and a small icon on the right. The terminal content shows a blue tilde character (~) on the first line, followed by a green prompt character (>) and the command 'npx nx migrate latest' on the second line.

Common Mistake

Ignoring Speed





Congestion

Result:

feat(create-orders): implement new flow and fix some issues
#15219

 Open `qcon:main` ← `heavycoder:new-order-flow-and-fix-bugs`   ▼

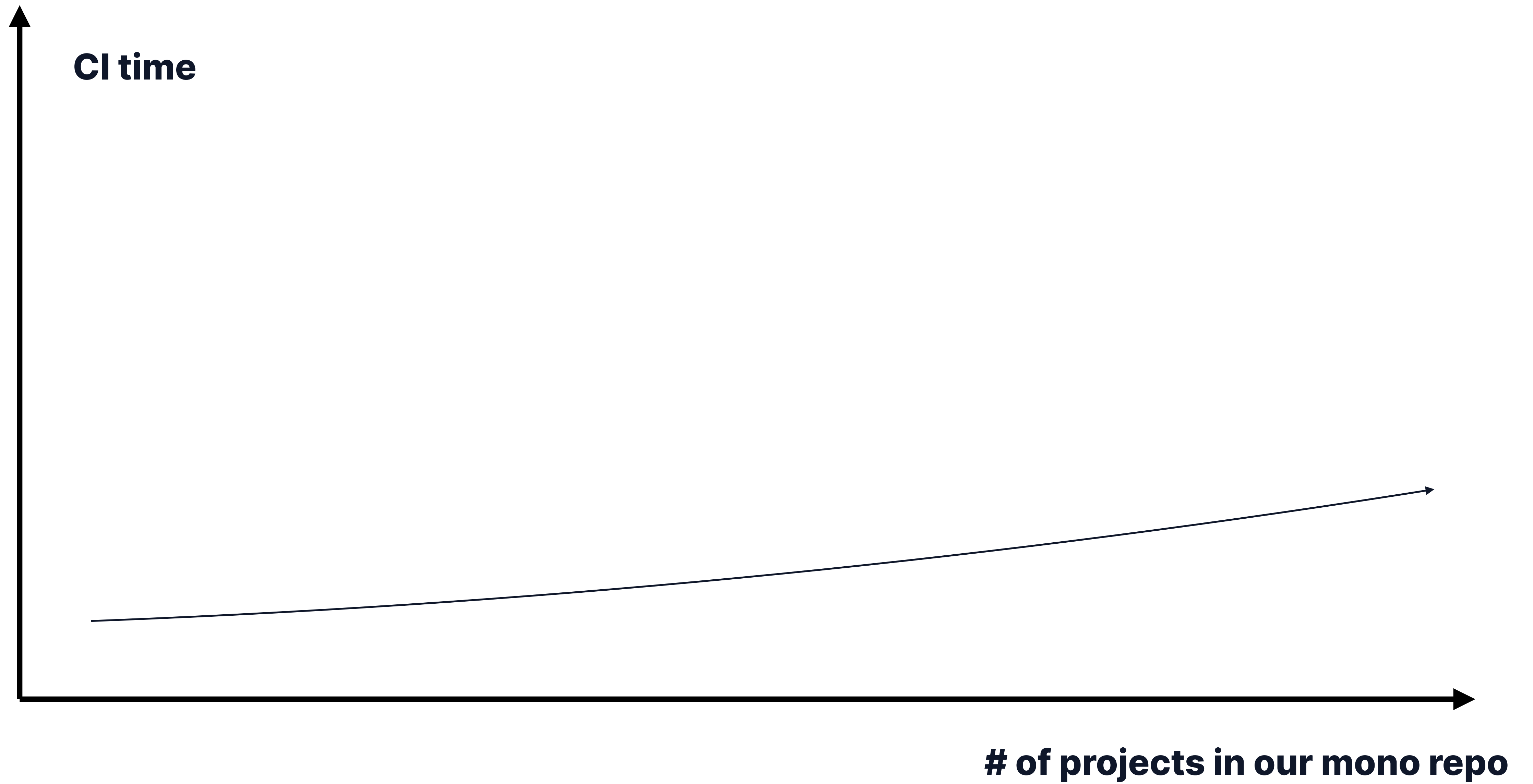
 Conversation 9

 Commits 5

 Checks 2

 Files changed 200





Layers of Speed

Run Tasks
across Machines

Remote Cache

Cache

Only “affected” Projects

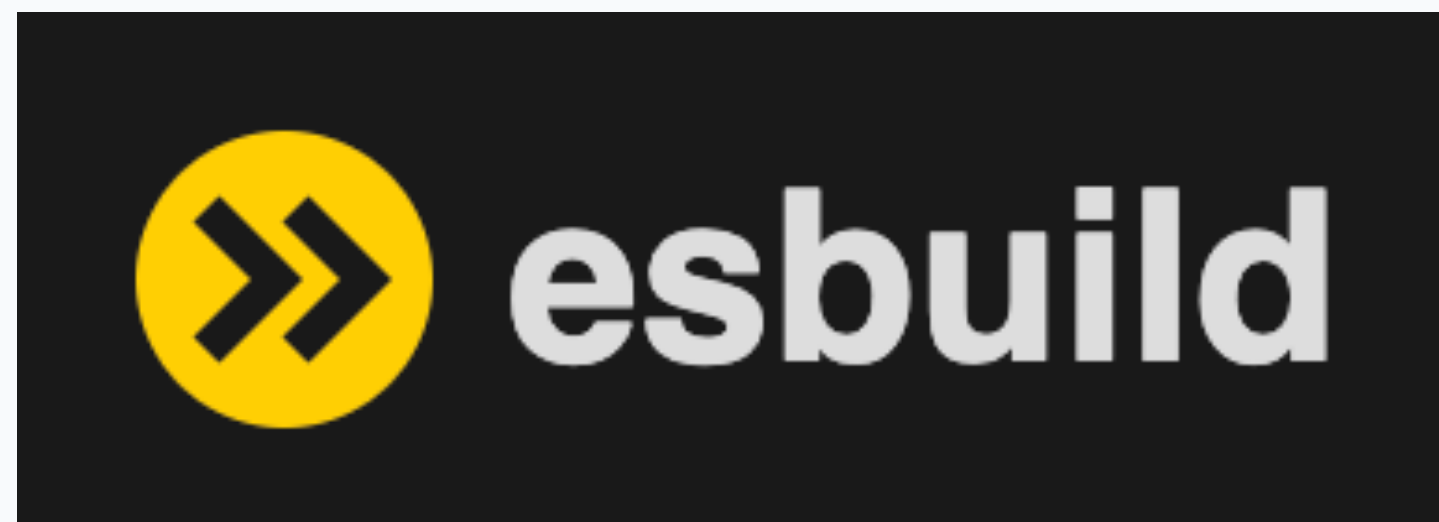
Intelligent Parallelisation

Fast (build) Tooling

Layers of Speed

Fast Tooling

Fast tooling



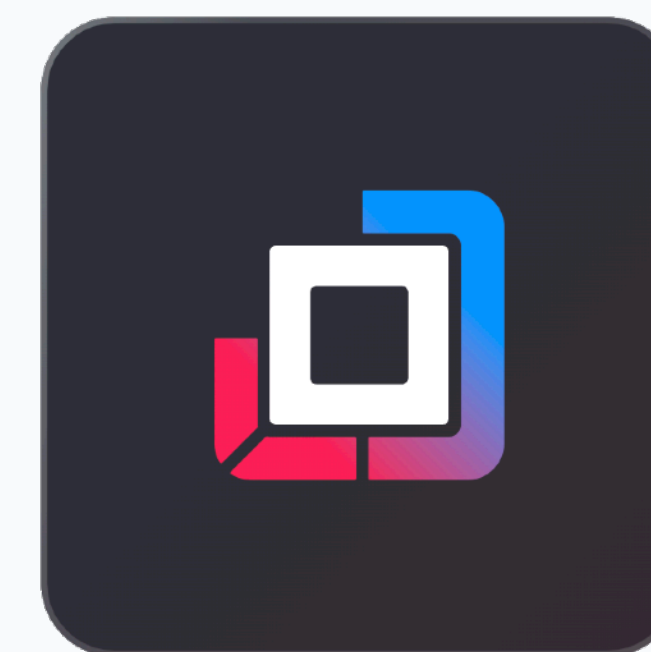
rspack



Vitest



Vite



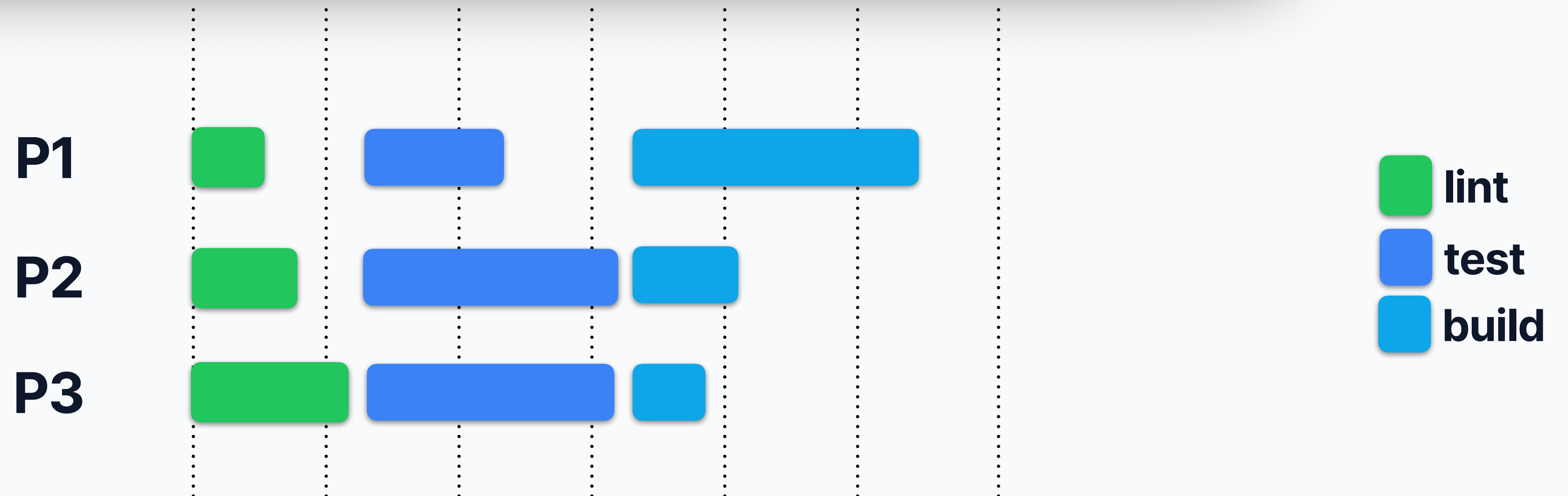
Turbopack

Layers of Speed

Intelligent Parallelization

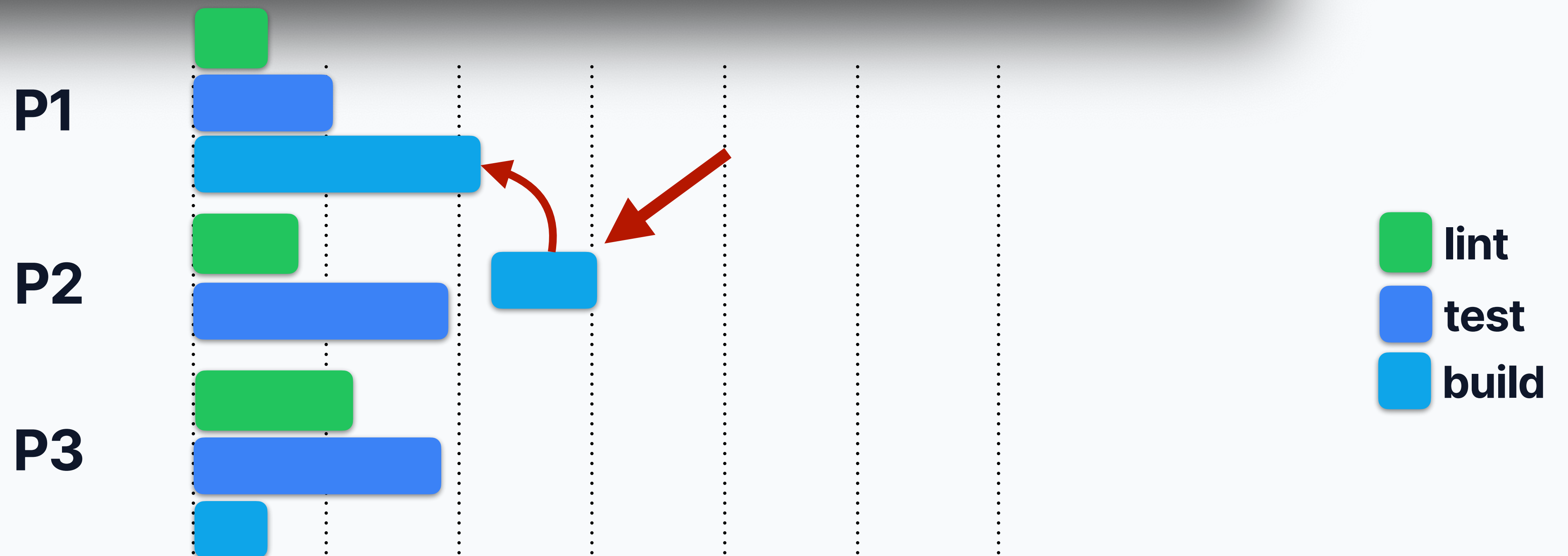
Intelligent Parallelization

```
~ . . . . .  
> npx nx run-many -target=lint  
npx nx run-many -target=test  
npx nx run-many -target=build
```



Intelligent Parallelization

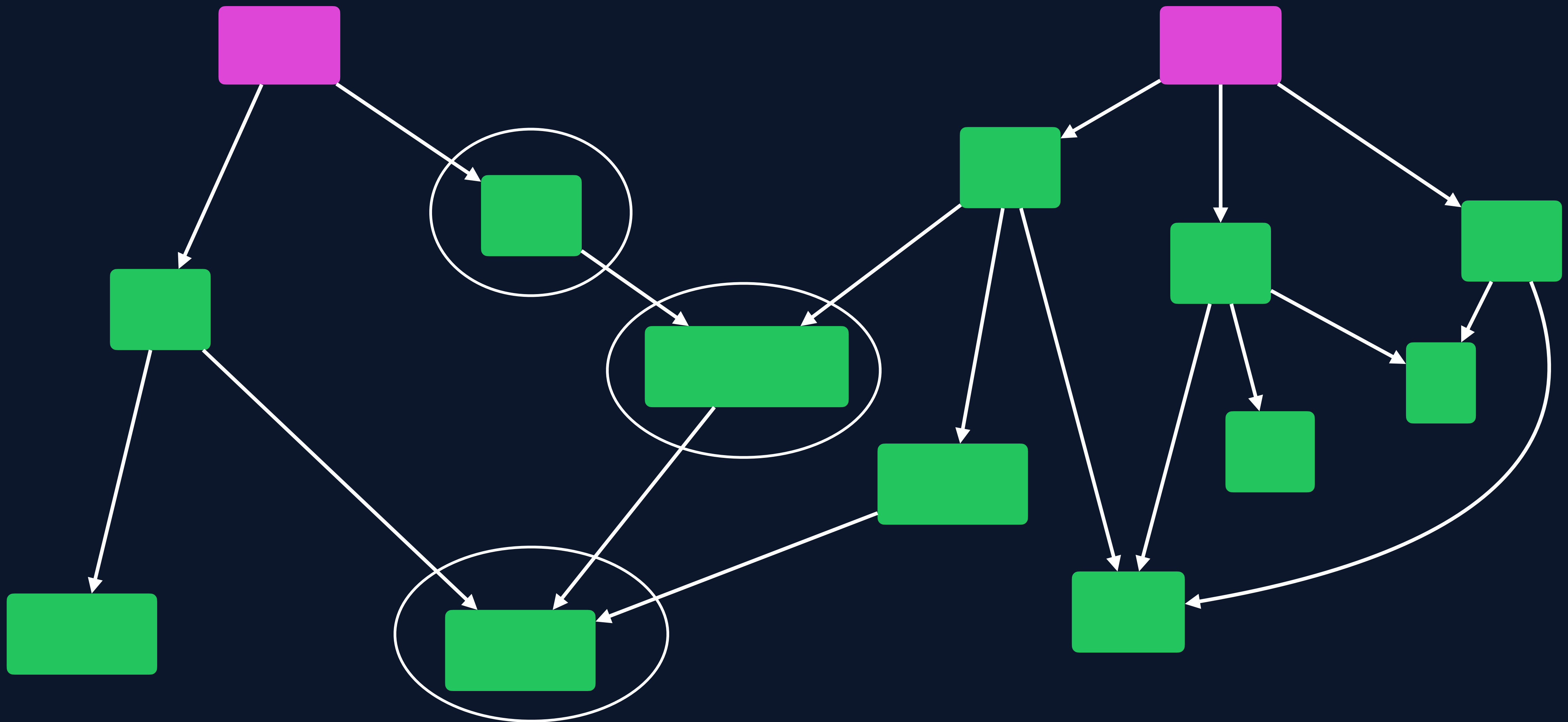
```
~ . . . . .  
> npx nx run-many -targets=lint,test,build
```





**How does
this work?**

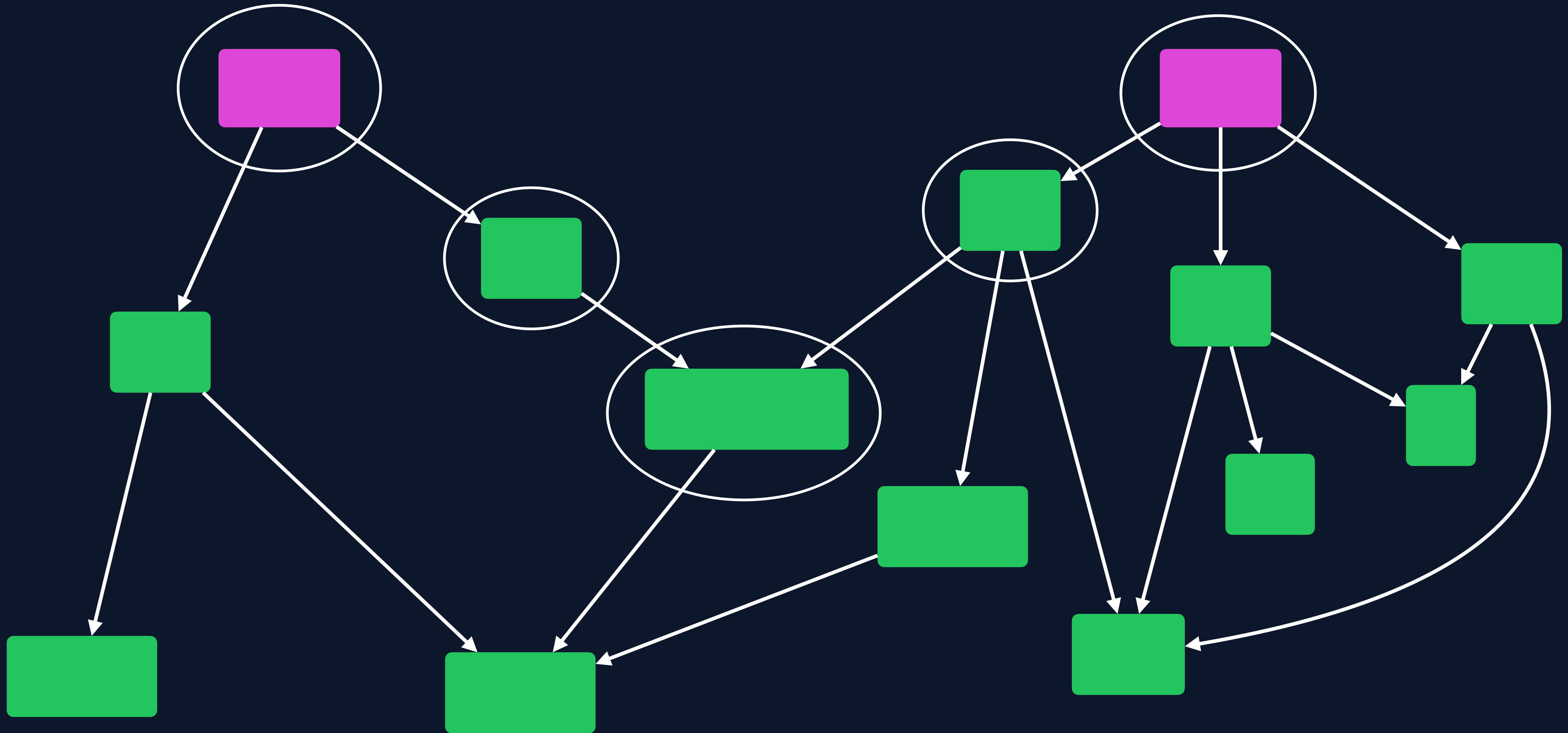
The Graph



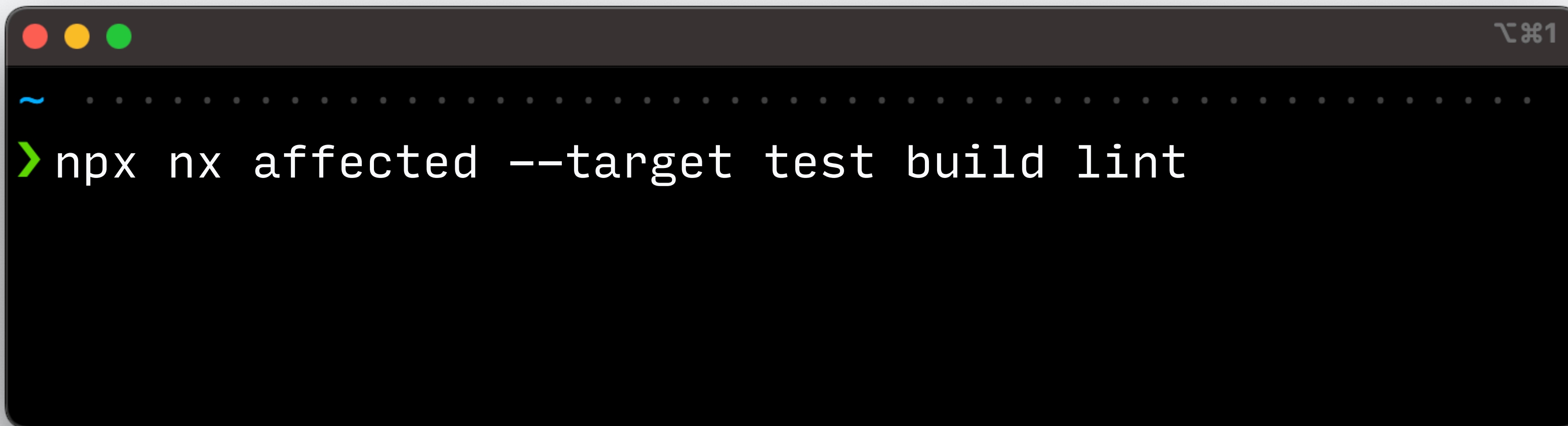
Layers of Speed

Only “affected” Projects

Affected Nodes only



Only run affected projects



```
~ . . . . .  
> npx nx affected --target test build lint
```

A terminal window with a dark background and a light gray title bar. The title bar contains three colored window control buttons (red, yellow, green) on the left and a window icon with the number '1' on the right. The terminal content shows a prompt character '~' followed by a dotted line, and a command prompt '>' followed by the command 'npx nx affected --target test build lint'.

Caching

**The Key Ingredients
for Fast CI**

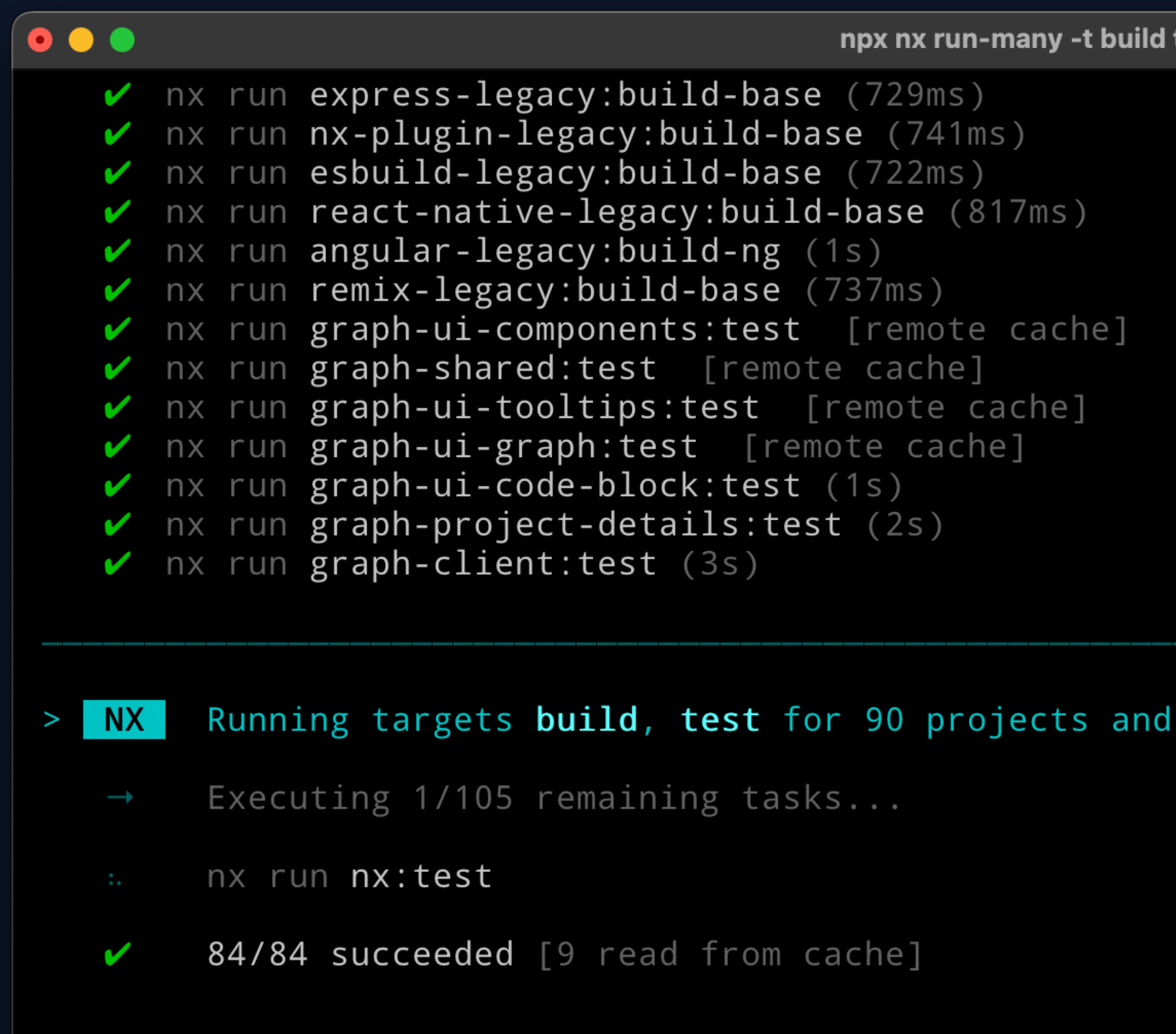
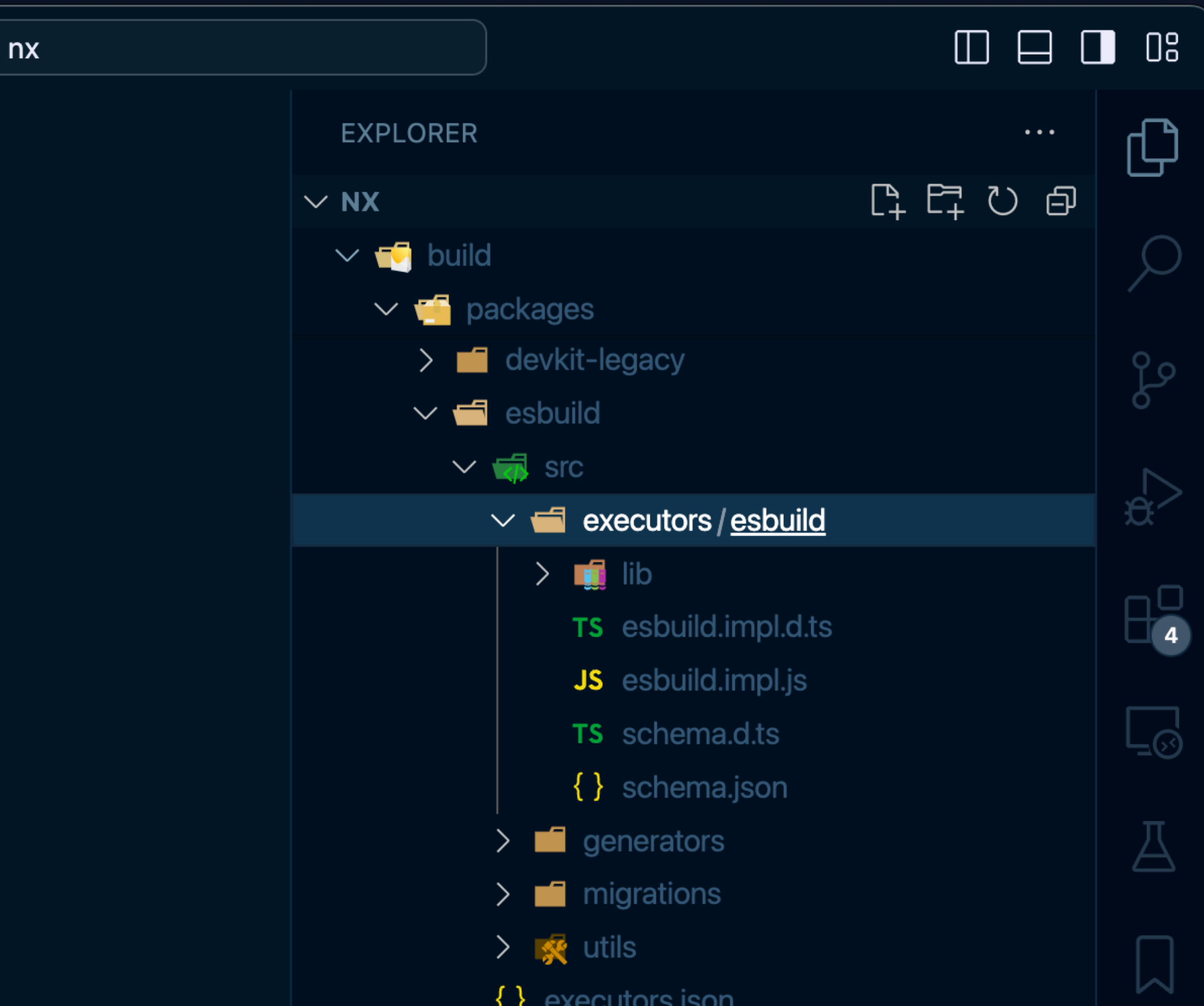
Distribution

Caching

**Never run the same
computation twice.**

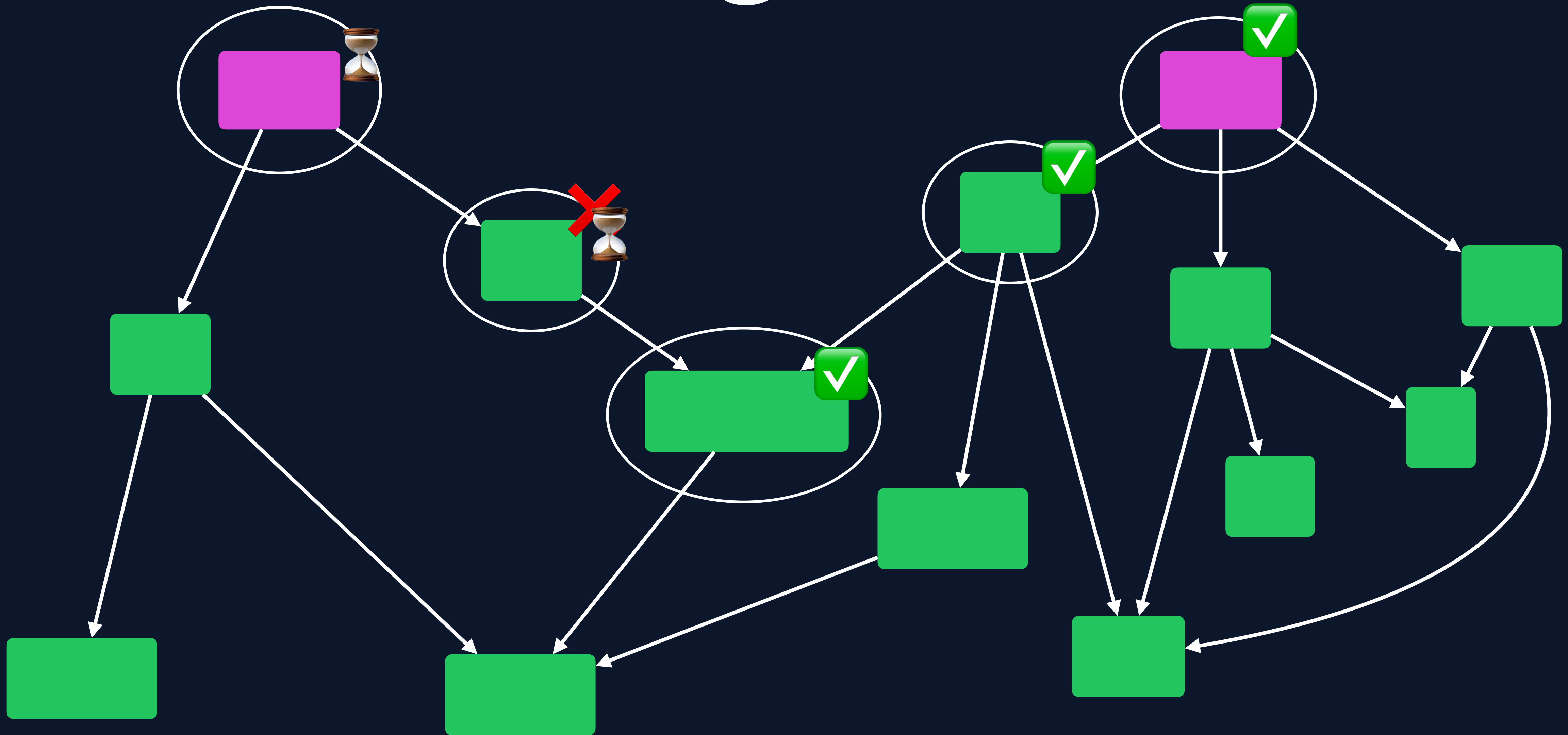
Built-in local and remote caching to speed up your tasks and save you time and money.

Caching

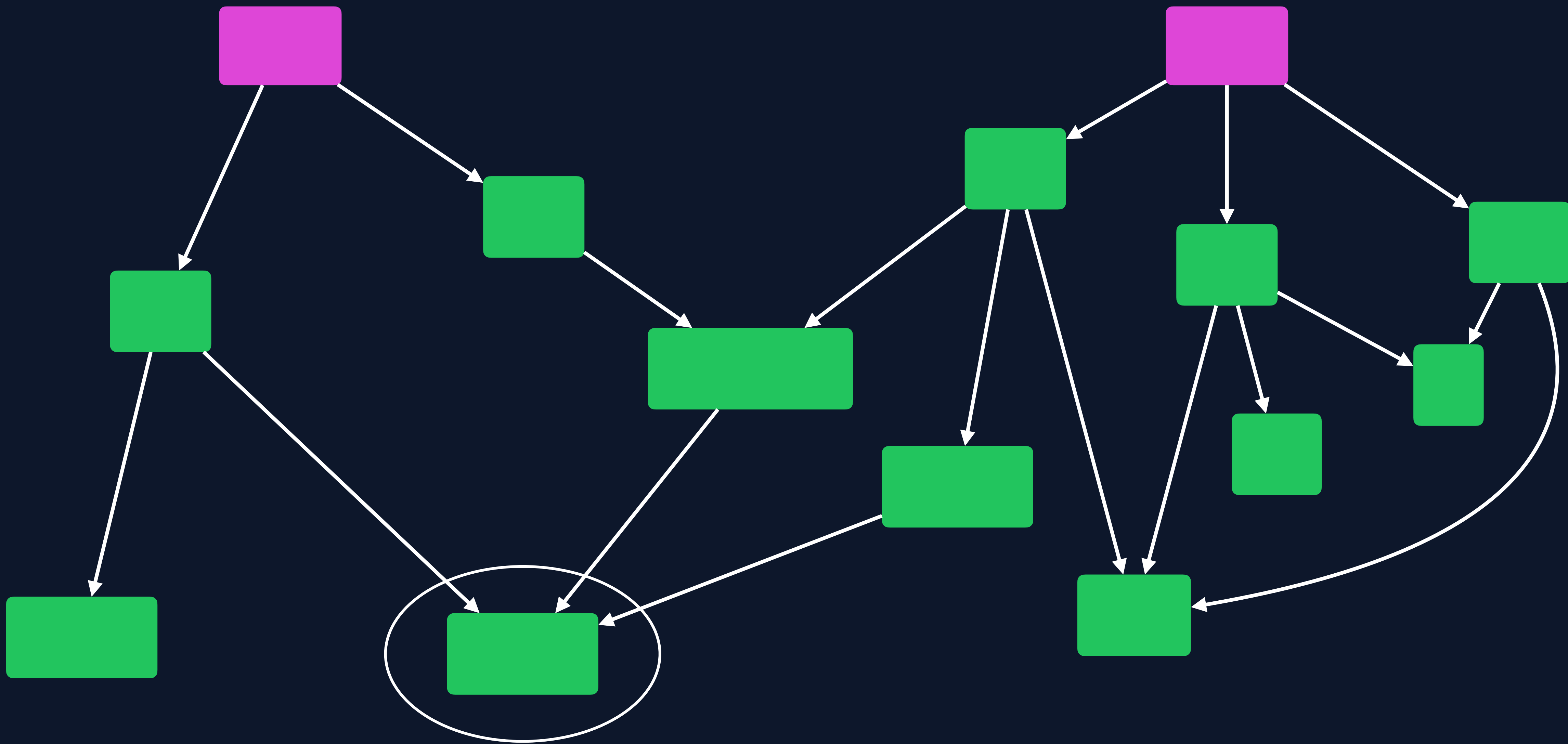


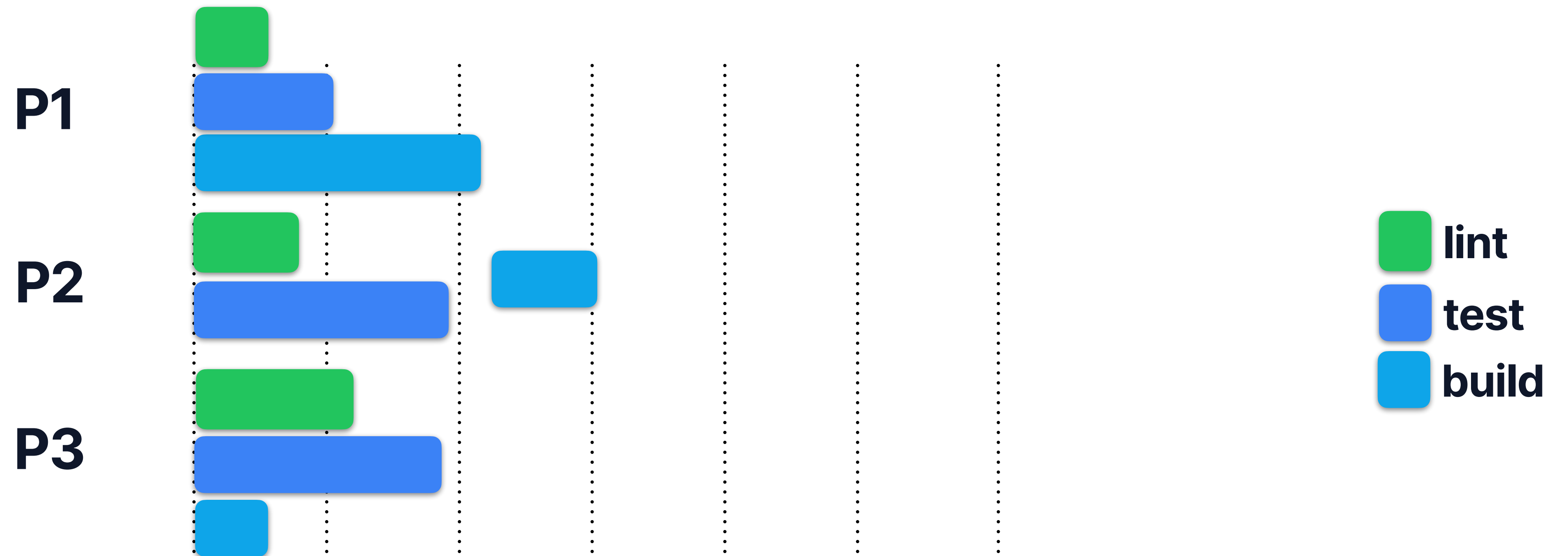
Imagine function **memoization**
but **applied to processes**

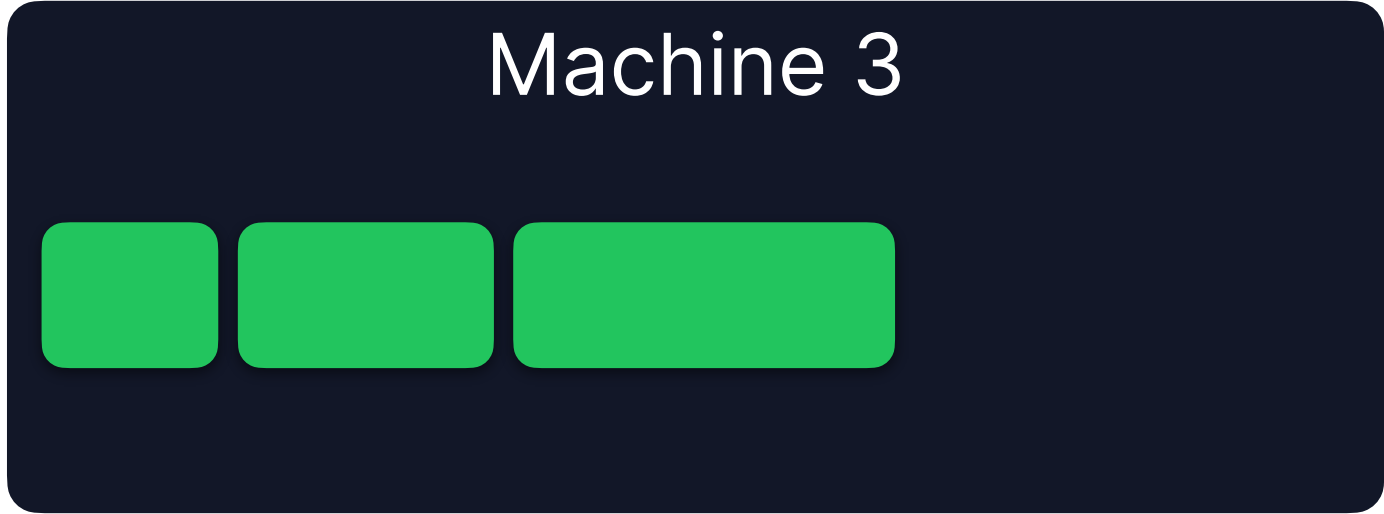
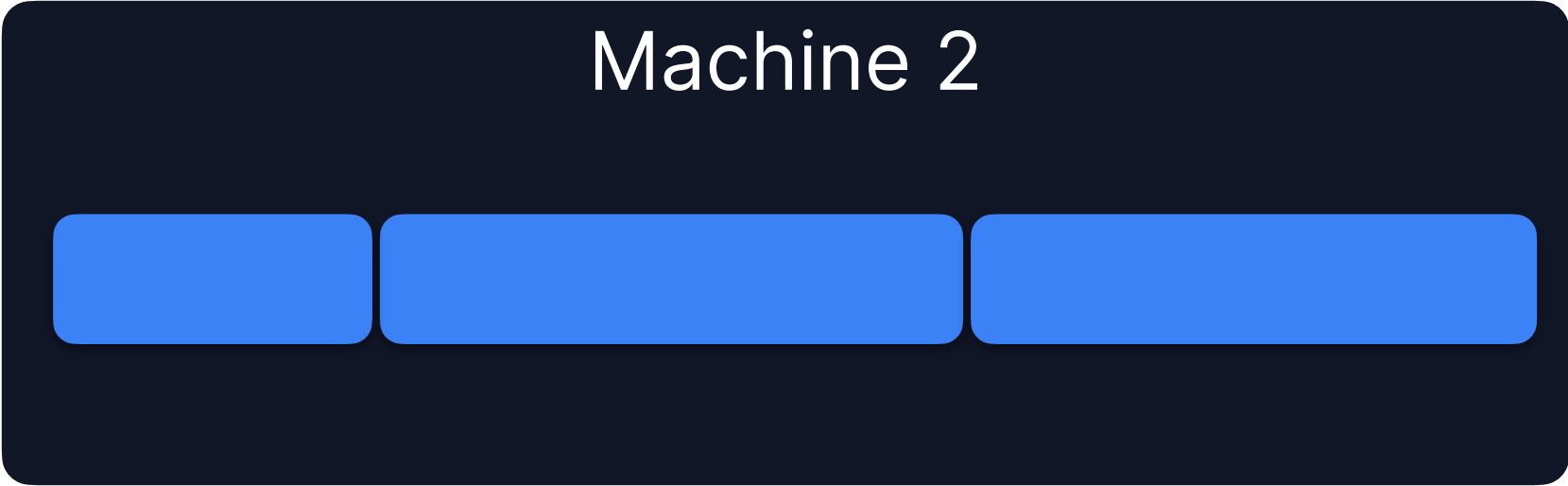
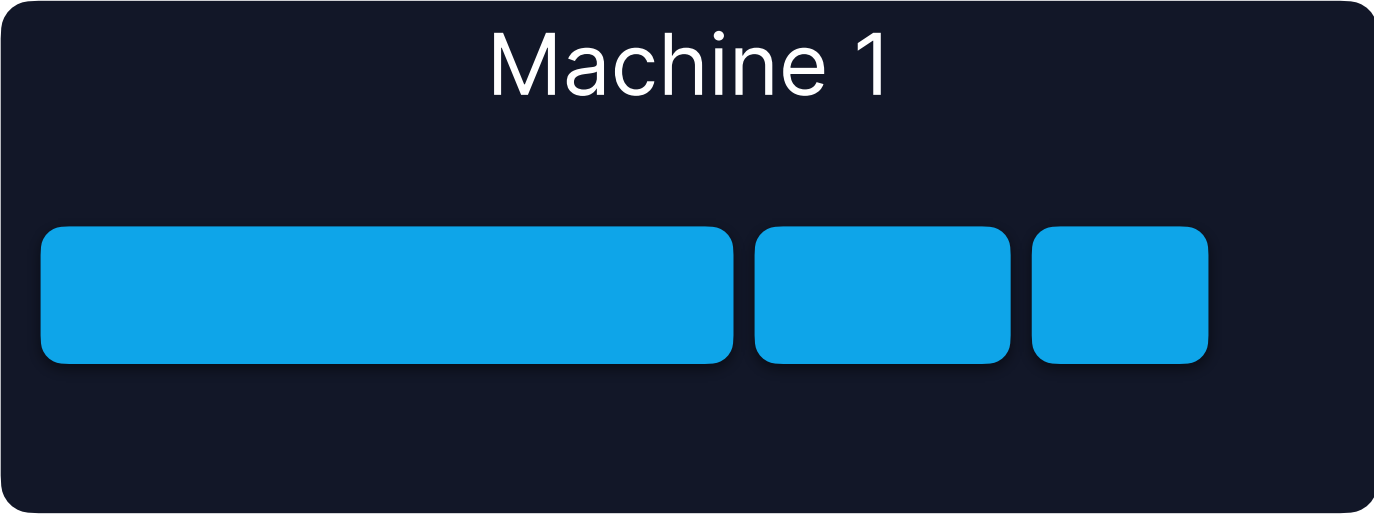
Caching in action

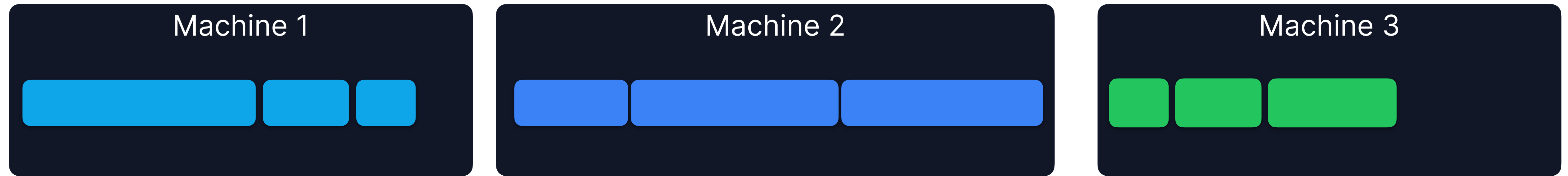


Distribution









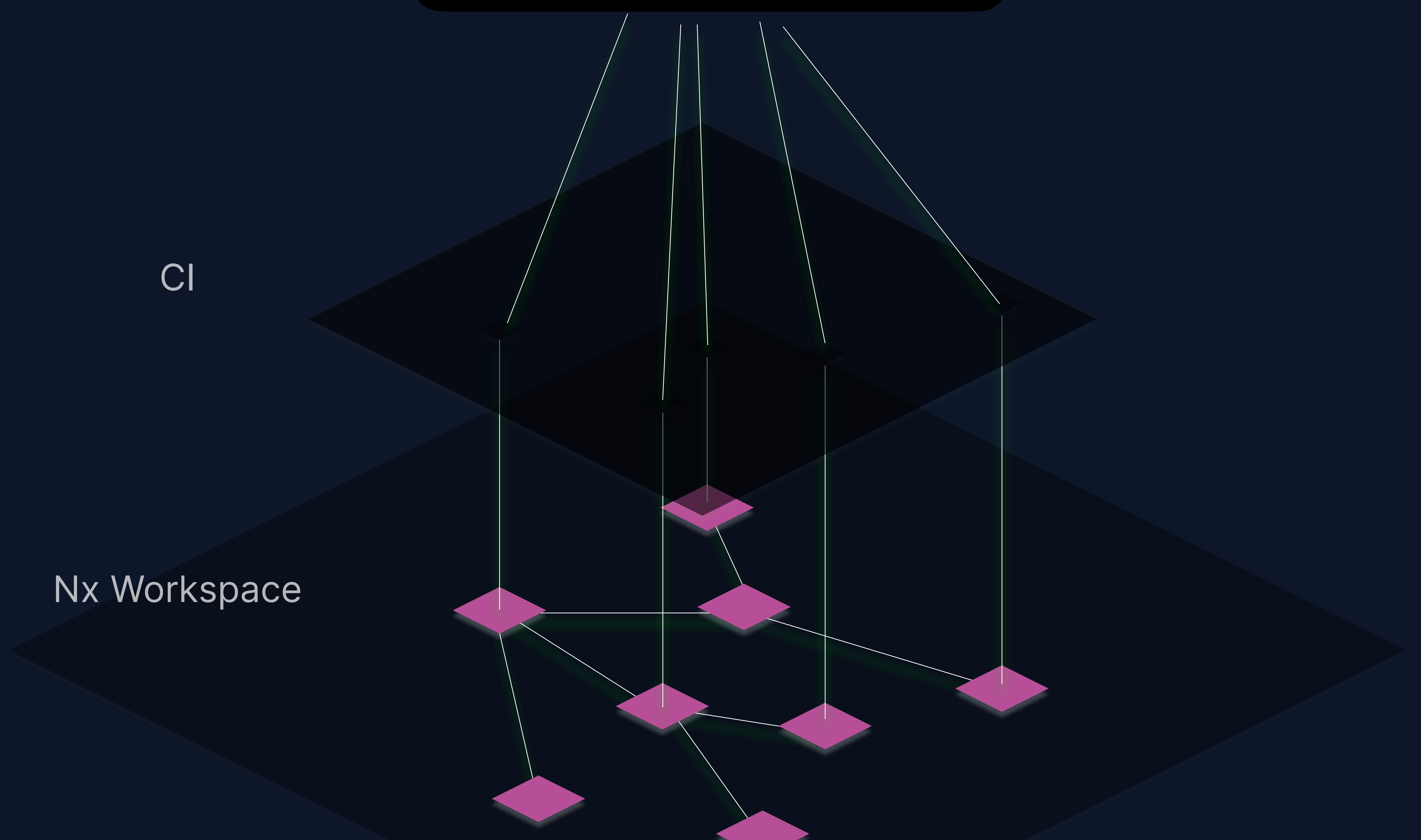
- Different running times leading to low efficiency
- How do we determine the number of machines?
- Number of machines is static
- Complexity into spinning up machines
- What happens as project structure changes

Nx Agents

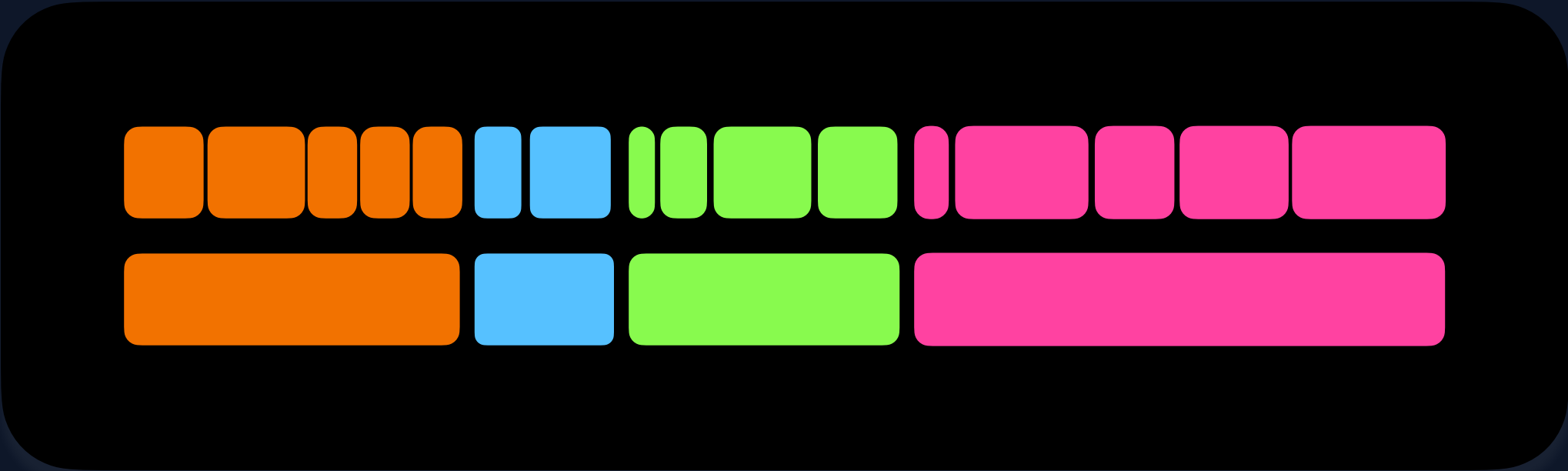
- Leverage the monorepo project graph knowledge
- You describe **what** to run
- Dynamic scaling
- Automated fine-grained distribution
- Automated flaky task rerunning

CI

Nx Workspace



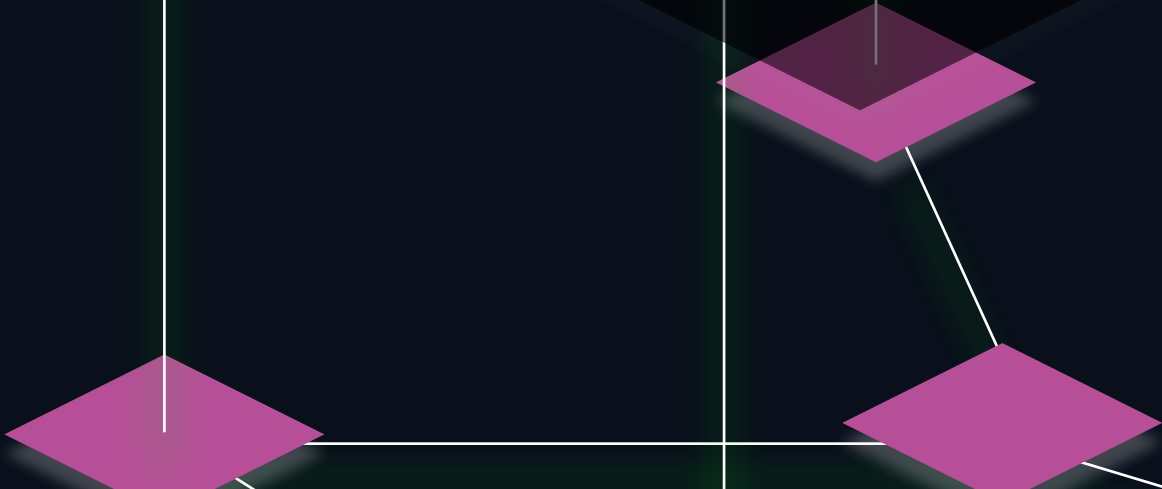
Nx Cloud



CI

```
~  
> nx affected -t build test lint e2e
```

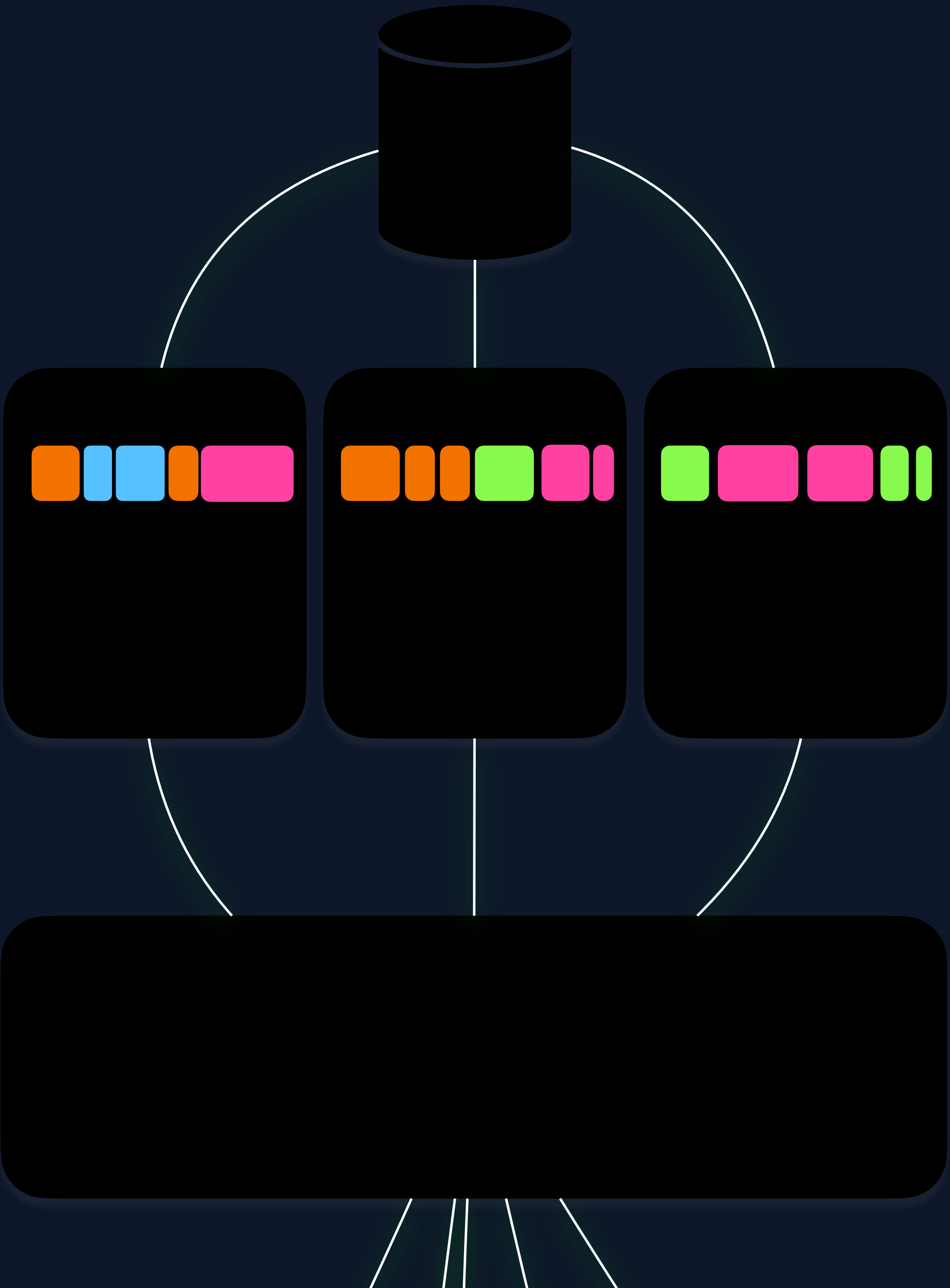
Nx Workspace



Remote Cache

Nx Agents

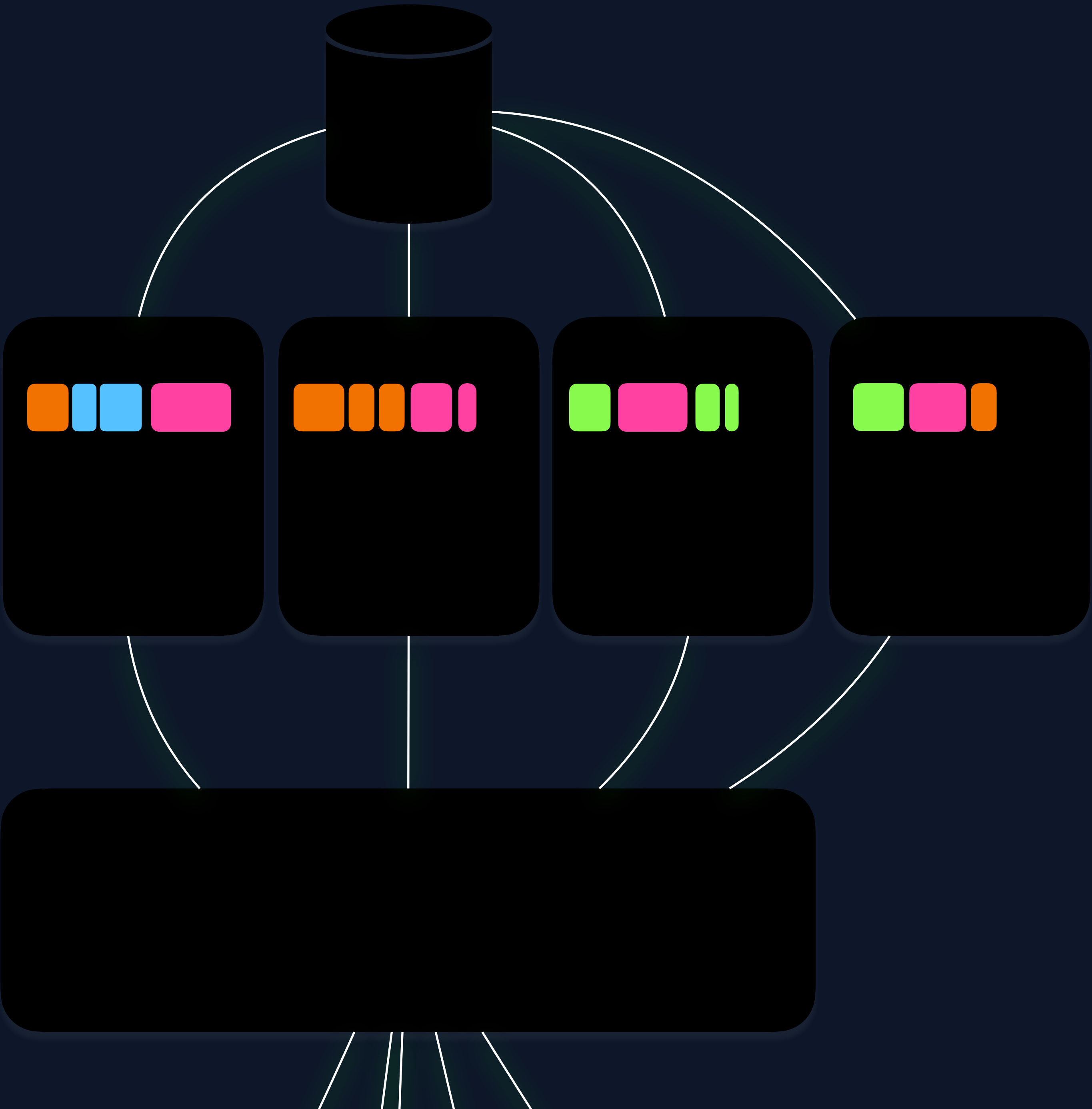
Nx Cloud



Remote Cache

Nx Agents

Nx Cloud



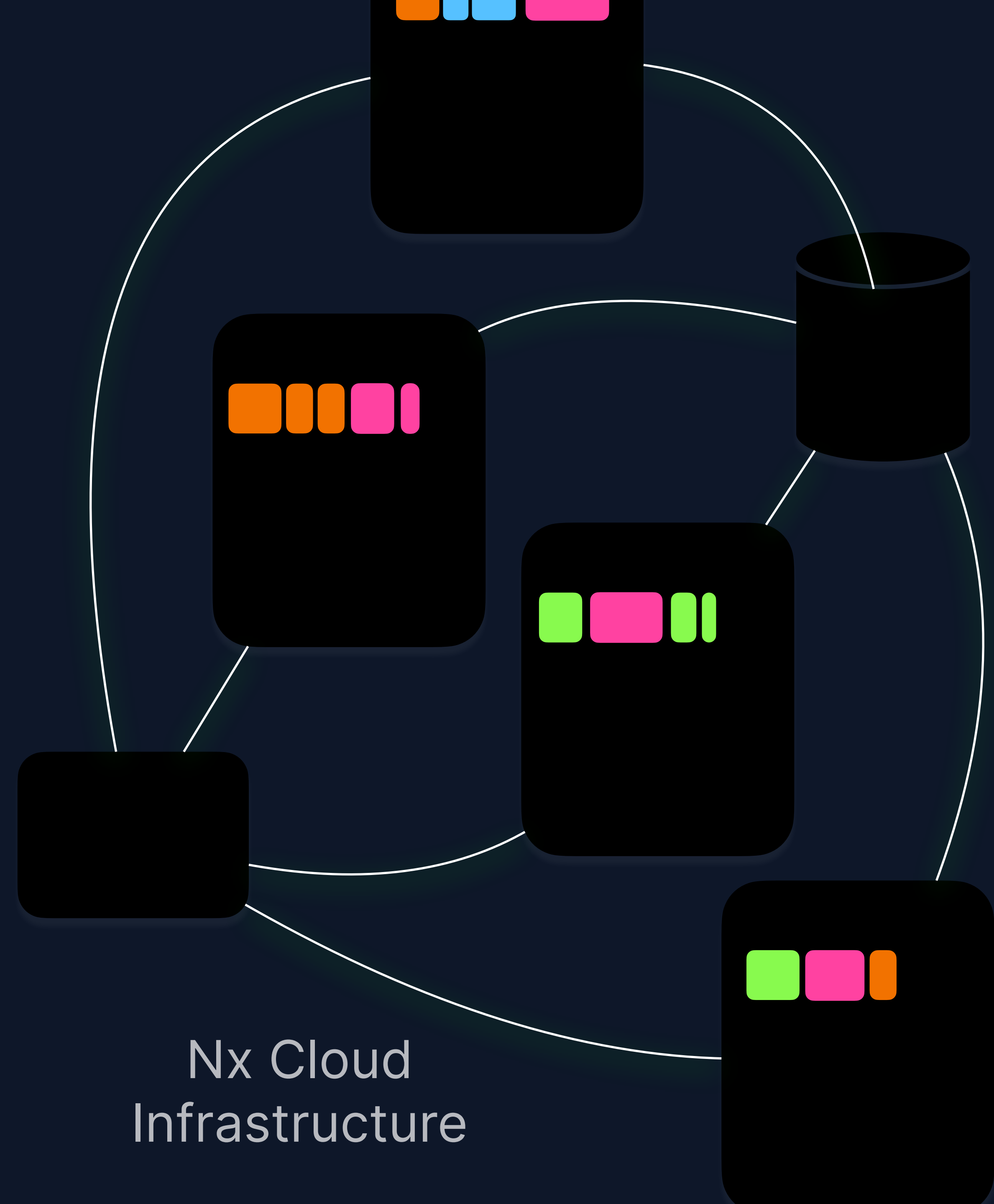
← → Search

ManagedAgentsDemo Untitled-1

```
runs-on: ubuntu-latest
timeout-minutes: 240
if: ${{ github.event_name == 'pull_request' }}
steps:
  - uses: actions/checkout@v2
    with:
      ref: ${{ github.event.pull_request.head.ref }}
      fetch-depth: 0
  - uses: actions/setup-node@v1
    with:
      node-version: '20'
  - name: Start CI run
    run: 'npx nx-cloud start-ci-run --distributes-on="15 linux-medium-plus-js" --stop-agents-after="e2e-ci"'
  - run: yarn install --frozen-lockfile
  - name: Run verification
    run: 'npx nx affected -t export test lint --parallel 1 && npx nx affected -t e2e-ci --parallel 1'
```

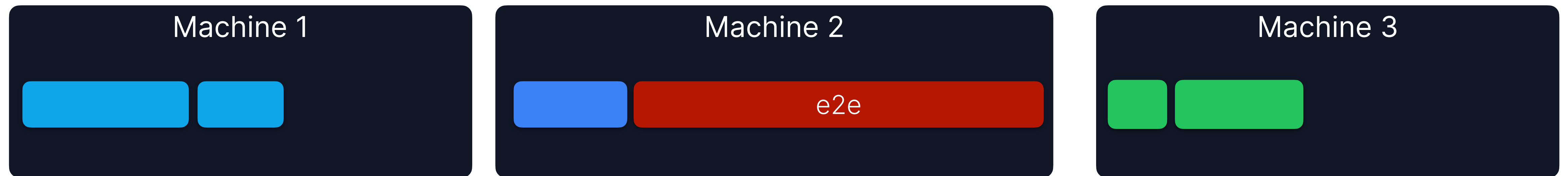
0 0 CodeStream Live Share -- VISUAL LINE -- 2 Ln 45, Col 1 (139 selected) Spaces: 2 UTF-8 LF YAML Prettier

Your CI



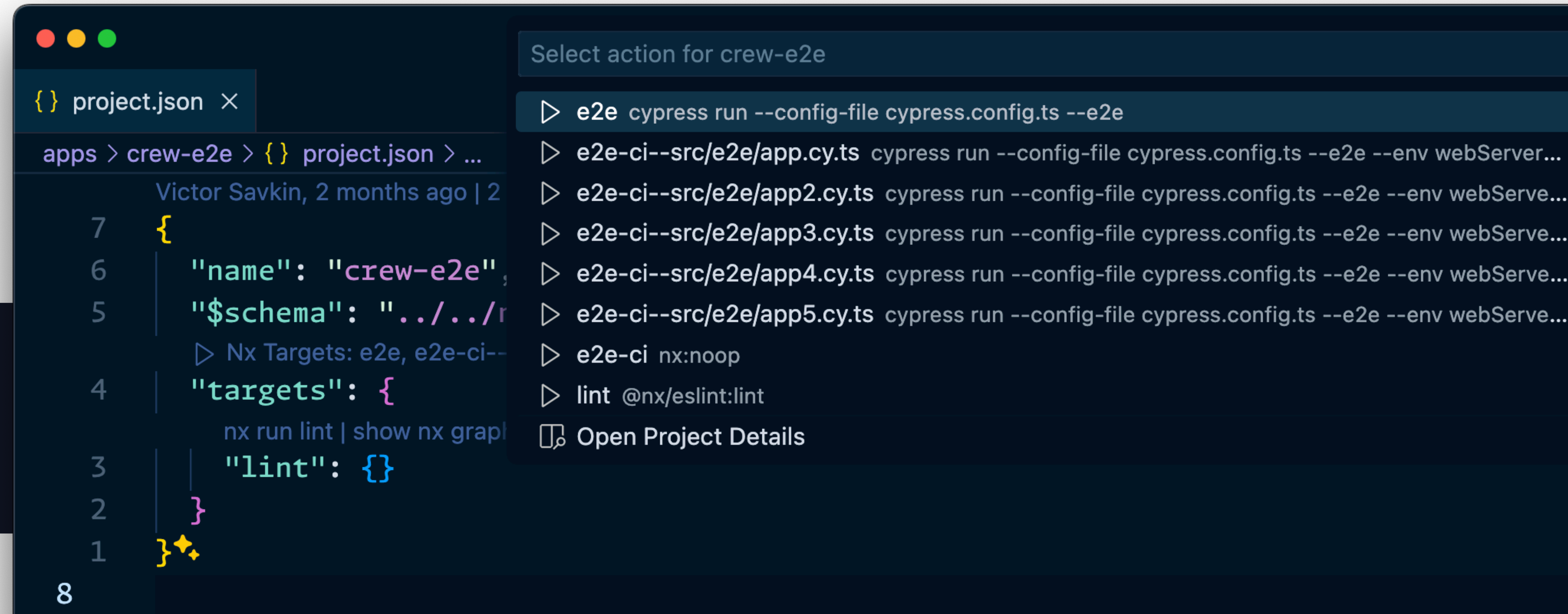
Nx Cloud
Infrastructure

Fine-grained Distribution



Fine-grained Distribution

Machine 1



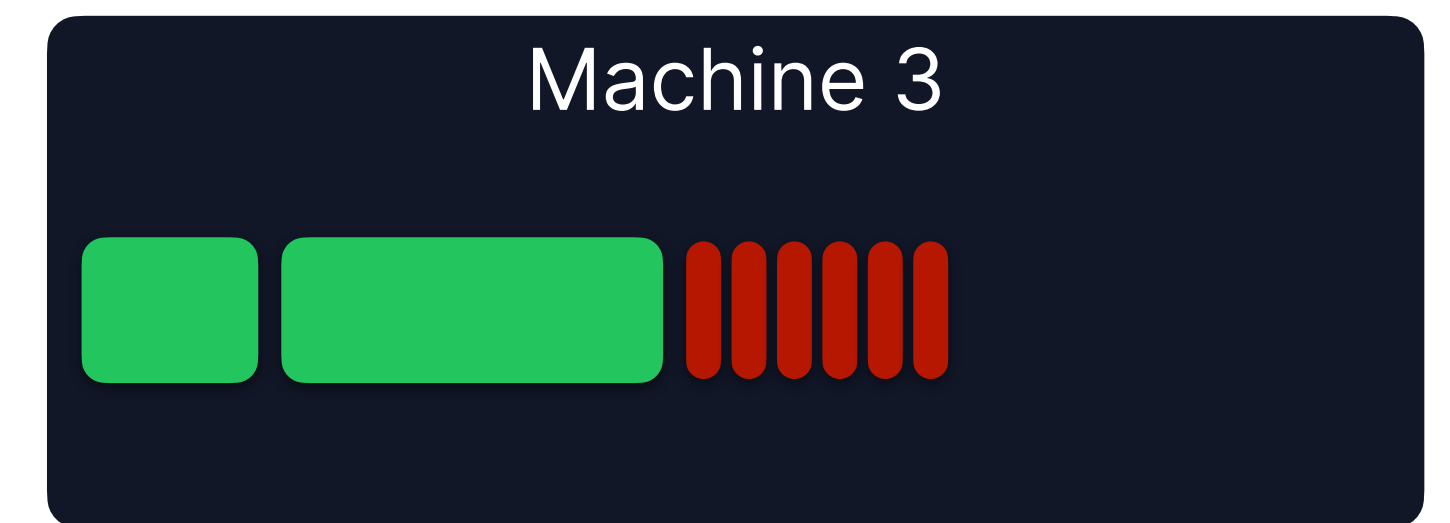
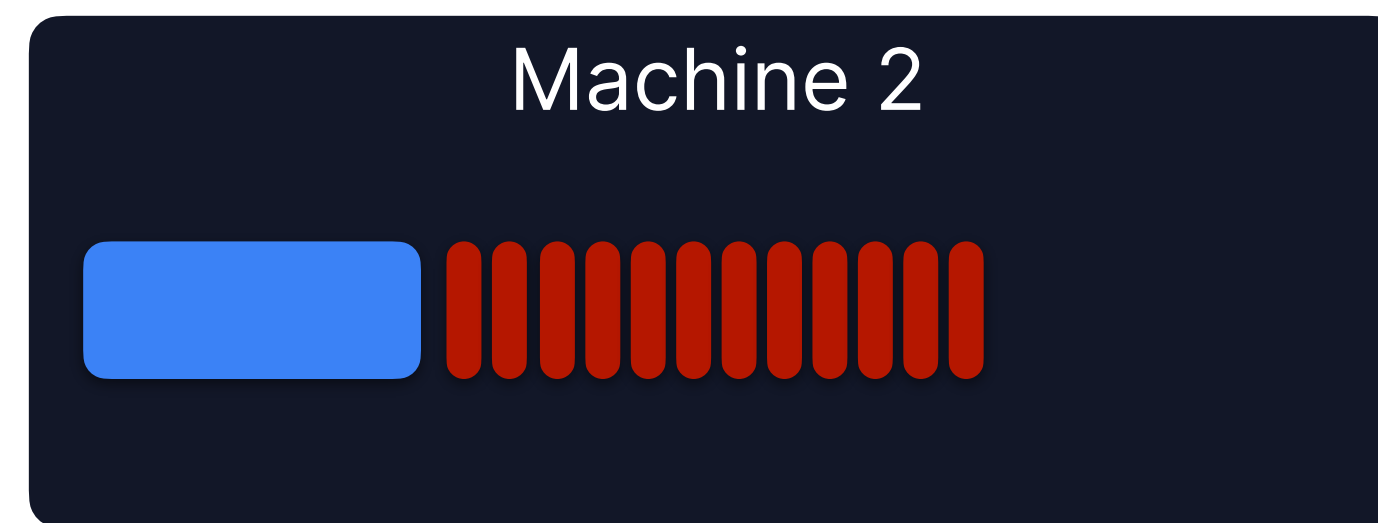
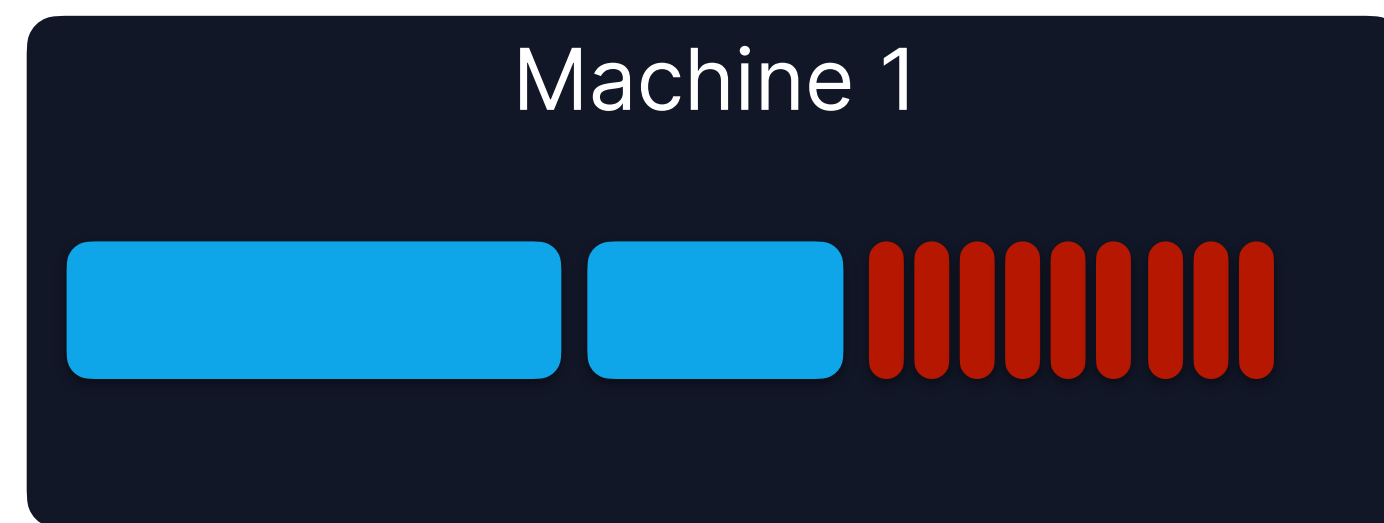
The screenshot shows a code editor with a file named `project.json` open. The file content is as follows:

```
7 {  
6   "name": "crew-e2e",  
5   "$schema": "../..",  
4   "targets": {  
3     "lint": {}  
2   }  
1 } ✨
```

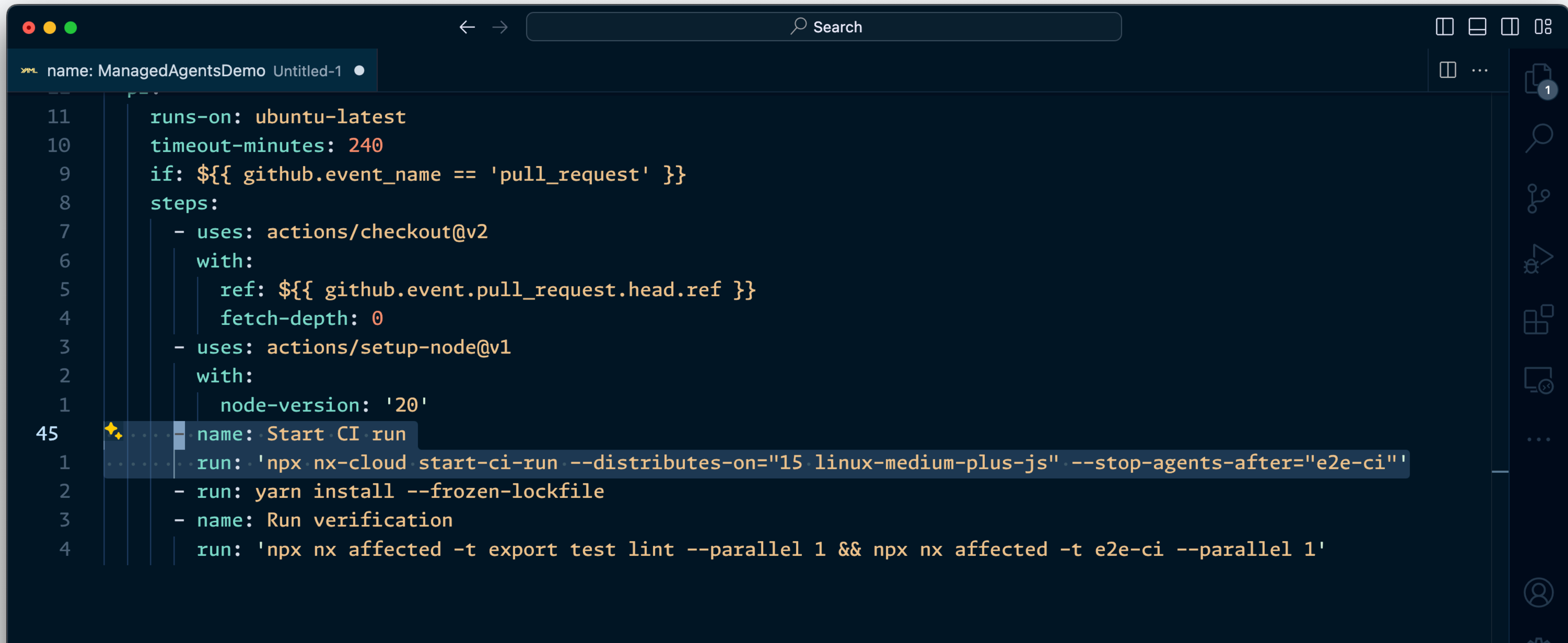
Below the file content, there is a command palette titled "Select action for crew-e2e". It lists several actions:

- ▶ e2e cypress run --config-file cypress.config.ts --e2e
- ▶ e2e-ci--src/e2e/app.cy.ts cypress run --config-file cypress.config.ts --e2e --env webServer...
- ▶ e2e-ci--src/e2e/app2.cy.ts cypress run --config-file cypress.config.ts --e2e --env webServe...
- ▶ e2e-ci--src/e2e/app3.cy.ts cypress run --config-file cypress.config.ts --e2e --env webServe...
- ▶ e2e-ci--src/e2e/app4.cy.ts cypress run --config-file cypress.config.ts --e2e --env webServe...
- ▶ e2e-ci--src/e2e/app5.cy.ts cypress run --config-file cypress.config.ts --e2e --env webServe...
- ▶ e2e-ci nx:noop
- ▶ lint @nx/eslint:lint
- 🔗 Open Project Details

Fine-grained Distribution



Enabling Distribution



```
11 runs-on: ubuntu-latest
10 timeout-minutes: 240
9 if: ${ github.event_name == 'pull_request' }
8 steps:
7   - uses: actions/checkout@v2
6     with:
5       ref: ${ github.event.pull_request.head.ref }
4       fetch-depth: 0
3   - uses: actions/setup-node@v1
2     with:
1       node-version: '20'
45  ✨ - name: Start CI run
1     run: 'npx nx-cloud start-ci-run --distributes-on="15 linux-medium-plus-js" --stop-agents-after="e2e-ci"'
2   - run: yarn install --frozen-lockfile
3   - name: Run verification
4     run: 'npx nx affected -t export test lint --parallel 1 && npx nx affected -t e2e-ci --parallel 1'
```



Nx Cloud

<https://nx.app>



OSS - MIT



Commercial
Add-on

**Wrapping
up...**

Smart Monorepos · Fast CI

Nx is a `build system` with `built-in tooling` and `advanced CI capabilities`. It helps you maintain and scale monorepos, both locally and on CI.

Structure

Automation

Throughput

Smart Monorepos · **Fast CI**

Structure

Not just a single monorepo

Breaking into smaller units

Domain Boundaries

Automation

Code generation

Automated upgrades

Key for adoption & scaling

Throughput

“Layers of Speed”

Caching

Distribution

Sounds interesting?



VoxxedDays Zurich

<https://www.youtube.com/watch?v=15VeTQLnWrs>

QCon
LONDON by InfoQ

Conference: March 27-29, 2023
The QEII Centre, London
In-person or online

SESSION

From Monorepo Mess to Monorepo Bliss: Avoiding Common Mistakes

HOST

Juri Strumpflohner
Sr. Director of Developer Experience @Nx



QCon London 2023

<https://www.infoq.com/presentations/monorepo-mistakes/>

youtube.com/@nxdevtools



Get in touch!



@juristr



@juristr



@juridev



juri.dev



nx.dev



go.nx.dev/community